

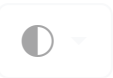
Due Sun, 11 Oct, 2026, 11:59 pm IST

Score: 0 / 35.5

Check all

Save

All scores



TDS 2026 Sep GA0

Instructions



Important: You **should not** take [TDS](#) if you score below 40% in GA0. This is your first *graded* assignment.

1. **Learn & solve.** To pass TDS, you need to learn lots of new stuff by yourself. This is your first *graded* assignment. If it's too hard, take TDS later.
2. **Use anything.** You can use any resources you want. The Internet, ChatGPT, friends, whatever. **Cheating** is allowed.
3. **It's hackable.** It's possible to get the answer to *some* questions by hacking the code for this quiz. **Hacking** is allowed.

Frequently Asked Questions

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Answer: Yes. GA0 is your first graded assignment, and its marks will contribute to your GAA average.

2 What happens if I am absent for GA0, score zero, or score below 40 marks? Will I be removed from the TDS course?

Answer: No. These outcomes do not automatically prevent you from continuing with the course. You may complete the remaining graded assignments and projects and can still pass the course, subject to meeting the overall course requirements.

3 What is GA0, why is it released early, and why should I attempt it before registering for the course?

Answer: GA0 is an early opportunity to familiarise yourself with the course content, level of difficulty, and expected workload. Attempting it before registration can help you assess whether you have sufficient time to take the course. If you score below 40% marks or find that the workload may be difficult to manage, please plan accordingly and consider taking TDS in a later term.



Have questions? [Join the discussion on Discourse](#)

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Recent saves Most recent (before deadline) is your official score

No recent saves

Questions

1. [Scale Manipulation Repair in Axis Design](#) (1 mark)
2. [Build a Binary Eval Rubric](#) (1 mark)
3. [The Bug Hunter \(Property-Based Testing\)](#) (1 mark)
4. [Calculate variance](#) (0.5 marks)
5. [Code Interpreter with AI Error Analysis](#) (1 mark)
6. [Fix the Color Encoding Mismatch](#) (1 mark)
7. [Count crawled HTML files](#) (1 mark)
8. [CSS: Featured-Sale Discount Sum](#) (2 marks)
9. [dbt: Operations performance mart](#) (1 mark)
10. [Write a FastAPI server to serve data](#) (3 marks)
11. [FastAPI Batch Sentiment Analysis](#) (1 mark)
12. [Get an LLM to say Yes](#) (2 marks)
13. [Create a GitHub Action](#) (1 mark)
14. [Reconstruct and desaturate an image](#) (1 mark)
15. [LLM Sentiment Analysis](#) (1 mark)

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

18. [Local Chat Endpoint](#) (3 marks)
19. [Replace across files](#) (2 marks)
20. [Sort and Filter a JSON Product Catalog](#) (0.5 marks)
21. [SQL: Average salary by department](#) (0.5 marks)
22. [Process files with different encodings](#) (2 marks)
23. [Use DevTools](#) (1 mark)
24. [Use GitHub](#) (1 mark)
25. [Deploy a POST analytics endpoint to Vercel](#) (3 marks)

- 1 Scale Manipulation Repair in Axis Design (1 mark)

Ask AI

- How can I detect whether this chart uses a truncated axis, dual-axis scaling trick, inverted axis, or incorrect log scale?
- Given the table, how do I compute a numeric distortion ratio that quantifies how misleading the current axis setup is?
- What minimal Chart.js code edits fix each scale manipulation type while preserving the data and chart structure?
- What quick checks should I run to confirm my corrected chart removes the deception and still renders correctly?

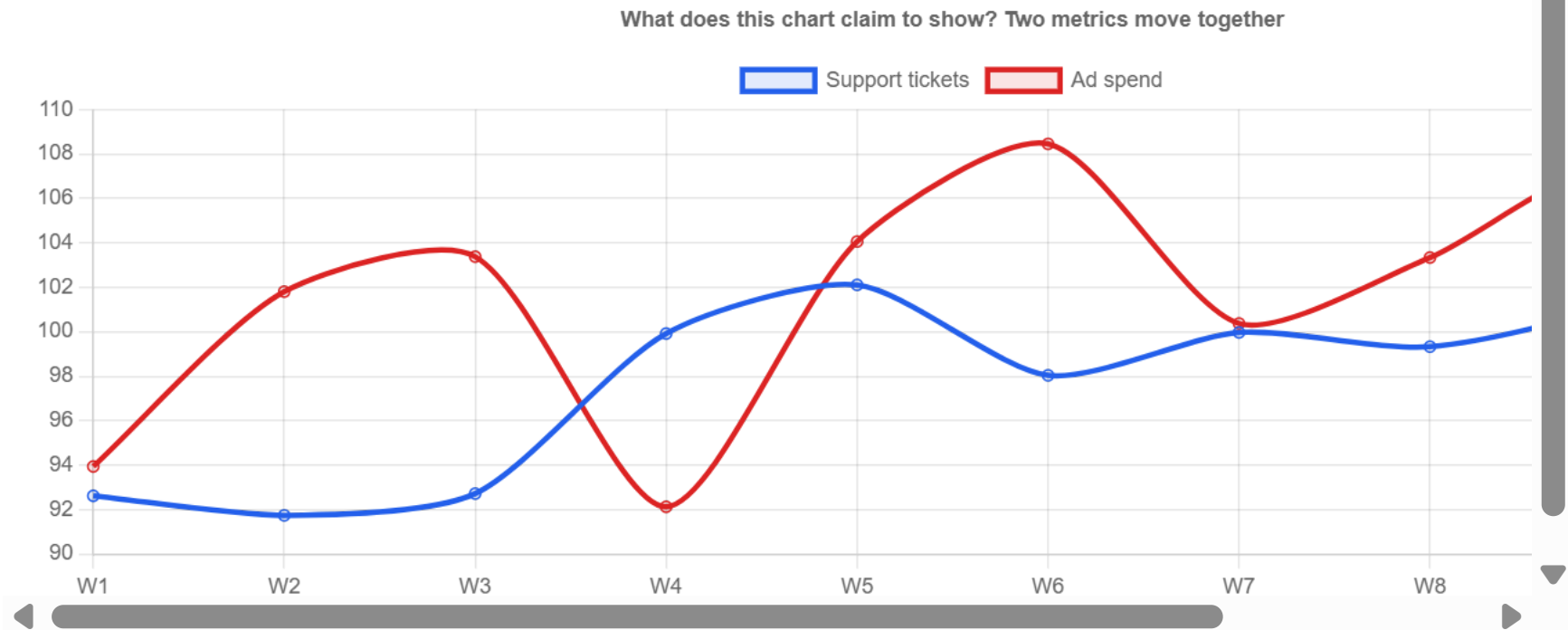
This chart uses a deceptive scale configuration. Your goal is to identify the manipulation, quantify the distortion, and submit corrected chart HTML.

Assigned chart: 2 of 20

What does this chart claim to show?

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

What does this chart claim to show? Two metrics move together

▼ Broken chart HTML (to fix)

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>What does this chart claim to show? Two metrics move together</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.js"></script>
  <style>

```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
</head>
<body>
  <h3>What does this chart claim to show? Two metrics move together</h3>
  <canvas id="chart"></canvas>
  <script>
    new Chart(document.getElementById('chart'), {"type":"line","data":{"labels":["W1","W2","W3","W4","W5","W6","W7"]
  </script>
</body>
</html>
```

▼ Underlying data table

Period	Series A	Series B
W1	92.62	321.83
W2	91.74	345.41
W3	92.72	350.14
W4	99.92	316.39
W5	102.11	352.19
W6	98.04	365.35
W7	99.97	341.12
W8	99.33	350.03
W9	101.48	361.66
W10	108.19	310.87

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

2. Fix the axis configuration and submit corrected HTML.
3. Write one sentence describing what the corrected chart reveals that the broken chart hid.

Include your explanation as an HTML comment at the top of your submission, for example: `<!-- Quantification: ... Distortion: ... -->`

Paste corrected HTML

```
</div>

<script>
const labels = ["W1","W2","W3","W4","W5","W6","W7","W8","W9","W10"];
const seriesA = [92.62, 91.74, 92.72, 99.92, 102.11, 98.04, 99.97, 99.33, 101.48, 108.19];
const seriesB = [321.83, 345.41, 350.14, 316.39, 352.19, 365.35, 341.12, 350.03, 361.66, 310.87];

// Normalize each series to % change from its first value (W1 = 0%)
const pctChange = arr => arr.map(v => +(((v - arr[0]) / arr[0]) * 100).toFixed(2));
const pctA = pctChange(seriesA);
const pctB = pctChange(seriesB);

new Chart(document.getElementById("chart"), {
  type: "line",
  data: {
    labels: labels,
    datasets: [
      { label: "Support tickets (% change)", data: pctA, borderColor: "#1a56db", backgroundColor:
"rgba(26,86,219,0.15)", tension: 0 },
      { label: "Ad spend (% change)", data: pctB, borderColor: "#d62828", backgroundColor: "rgba(214,40,40,0.15)",
tension: 0 }
    ]
  },
},
```

Error: Distortion description phrase missing. Include a clear manipulation description in your comment.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

2 Build a Binary Eval Rubric (1 mark)

Ask AI

- How do I convert a vague rubric into binary checks that are judgeable from output alone?
- What makes a binary check degenerate or weak in LLM-as-judge evaluations?
- How can I design checks that correlate with quality labels instead of style preferences?

Architecture: Client-side with AIPipe (LLM-as-judge validating the judge).

You're building an automated grader for **prompt quality for extracting structured output**. An LLM will evaluate student submissions using your rubric.

The problem with vague rubrics: "Is this code good?" gives inconsistent results. Your job is to decompose quality into binary (YES/NO) checks that an LLM can answer reliably.

Your task:

1. Examine the 3 example outputs below (marked GOOD, MEDIOCRE, POOR).
2. Write exactly **5** binary checks that collectively distinguish good from poor work.
3. Each check must be answerable YES/NO by reading the output alone.
4. Format: one check per line, as a complete question ending in "?".

Example outputs:

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Example:

Input: 'Ada from OpenAI moved to London'

Output: {"people":["Ada"],"orgs":["OpenAI"],"locations":["London"]}

MEDIOCRE: **Extract** entities and **return** JSON **with** people, orgs, and locations. **Keep** it concise.

POOR: **Find** entities **in** text.



Before you start: evaluating this answer will make about **100 AIPipe API calls** (5 checks x 20 hidden examples), with up to **8** calls in parallel for speed. Make sure you have enough credits. Recommended: attempt this question near the end of the assignment after finishing lower-cost questions.

Submit your 5 binary checks, one per line.

Binary rubric checks

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

...



Evaluation: each check is run against 20 hidden examples; a check passes if correlation with ground truth is greater than 0.7 and it is not degenerate (always YES or always NO). Threshold: at least 4 checks must pass. This validator executes calls concurrently to reduce waiting time.

[Check](#)[Discuss](#)[Ask AI](#)

3 The Bug Hunter (Property-Based Testing) (1 mark)

Ask AI

- What makes property-based tests different from unit tests with hand-picked examples?
- How do I design a good property/invariant before writing Hypothesis strategies?
- How can I avoid writing weak Hypothesis tests that never hit the bug region?

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Architecture: In-browser Pyodide execution. Your submitted Python code is executed against both a buggy implementation and a reference implementation without calling any external runner URL.

Hand-crafted unit tests often miss edge cases. Your goal is to write a **Hypothesis** property test that automatically discovers a counterexample for this seeded function variant: **Billing Date Parser**.

Buggy function

```
from datetime import datetime

def parse_billing_date(date_str):
    dt = datetime.strptime(date_str, "%Y-%m-%d")
    if dt.month == 2 and dt.day == 29 and dt.year % 4 == 0:
        return datetime(dt.year, dt.month, 28)
    return dt
```

What the function should do

Parse a YYYY-MM-DD date string into a datetime representing the exact same calendar date.

Known passing unit tests (these do not catch the bug)

```
def test_regular_dates():
    assert parse_billing_date("2023-03-01").day == 1
    assert parse_billing_date("2023-12-31").month == 12
    assert parse_billing_date("2021-02-28").day == 28
```

Your task

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

3. We run your test for up to 1000 generated examples. You pass only if your property fails on buggy code but passes on the reference implementation.

► **Hint**

Submit complete Python test code

```
from hypothesis import given, strategies as st

@given(...)
def test_property(...):
    ...
```

Include imports and at least one `test_*` function decorated with `@given(...)`.

Assessed skills:

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- Writing robust behavioral properties
- Distinguishing buggy vs correct implementations

Check

Discuss

 Ask AI

4 Calculate variance (0.5 marks)

Spreadsheet: Excel, Google Sheets

You'll use spreadsheets for data cleaning and exploration. The most popular spreadsheet program is [Microsoft Excel](#) followed by [Google Sheets](#).

You may be already familiar with these. If not, make sure to learn the basics of both.

Go through the [Microsoft Excel video training](#) and make sure you cover:

- [Intro to Excel](#)
- [Rows & columns](#)
- [Cells](#)
- [Formatting](#)
- [Formulas & Functions](#)
- [Tables](#)
- [PivotTables](#)

Watch this video for an introduction to **Google Sheets** (49 min):

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



Ask AI

- What's the difference between mean, variance, and standard deviation, and when is each useful?
 - How do I calculate population variance step by step from a list of numbers?
 - How can I avoid confusing sample variance with population variance in Excel, Python, or SQL?
-

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

PrecisionParts Manufacturing is a leading supplier of precision-engineered components for the automotive and aerospace industries. Quality control is paramount in their operations, as even small variations in part dimensions can lead to product failures, costly recalls, and safety issues. The company has implemented a rigorous statistical process control system to monitor manufacturing consistency.

The quality control team measures critical dimensions of manufactured parts throughout each production run. To ensure process stability, they need to calculate the variance of these measurements - a key statistical measure that indicates how much the individual measurements deviate from the average. High variance suggests the manufacturing process may be unstable and requires immediate attention.

This variance analysis helps the team:

- **Detect process drift:** Sudden increases in variance can indicate machinery calibration issues or tool wear
- **Maintain quality standards:** Parts with high dimensional variance may not meet customer specifications
- **Predict maintenance needs:** Variance trends can help schedule preventive maintenance
- **Optimize production parameters:** Understanding variance helps fine-tune machine settings

Your Task

Download [q-calculate-variance.json](#). It contains an array of 1000 measurements from the latest production run.

What is the sample variance of these measurements?

Calculate the **sample variance** (using N-1 denominator) using Python, Excel, or JavaScript. The data contains no missing values. Write the answer rounded to 2 decimal places, e.g. **123.45**.

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)[Check](#)[Discuss](#)[Ask AI](#)

5 Code Interpreter with AI Error Analysis (1 mark)

Large Language Models

This module covers the practical usage of large language models (LLMs).

LLMs incur a cost. For the May 2025 batch, use aipipe.org as a proxy. Emails with [@ds.study.iitm.ac.in](mailto:ds.study.iitm.ac.in) get a **\$1 per calendar month** allowance. (Don't exceed that.)

Read the [AI Pipe documentation](#) to learn how to use it. But in short:

1. Replace `OPENAI_BASE_URL`, i.e. `https://api.openai.com/v1` with `https://aipipe.org/openrouter/v1...` or `https://aipipe.org/openai/v1...`
2. Replace `OPENAI_API_KEY` with the [AIIPIPE_TOKEN](#)
3. Replace model names, e.g. `gpt-4.1-nano`, with `openai/gpt-4.1-nano`

For example, let's use [Gemini 2.0 Flash Lite](#) via [OpenRouter](#) for chat completions and [Text Embedding 3 Small](#) via [OpenAI](#) for embeddings:

```
curl https://aipipe.org/openrouter/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "openai/gemini-2.0-flash-lite",  
  "messages": [  
    {  
      "role": "user",  
      "content": "Hello!"  
    }  
  ],  
  "stream": false  
'
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
"messages": [{ "role": "user", "content": "What is 2 + 2?" } ]
}'
```

```
curl https://aipipe.org/openai/v1/embeddings \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $AIPPIPE_TOKEN" \
-d '{ "model": "text-embedding-3-small", "input": "What is 2 + 2?" }'
```

Or using [llm](#):

```
llm keys set openai --value $AIPPIPE_TOKEN

export OPENAI_BASE_URL=https://aipipe.org/openrouter/v1
llm 'What is 2 + 2?' -m openrouter/google/gemini-2.0-flash-lite-001

export OPENAI_BASE_URL=https://aipipe.org/openai/v1
llm embed -c 'What is 2 + 2' -m 3-small
```

For a 50% discount (but slower speed), use [Flex processing](#) by adding `service_tier: "flex"` to your JSON request.

AI Pipe

Anyone with a `study.iitm.ac.in` email can get a free API key from [aipipe.org](#) and use up to **\$2 per calendar month** for this course. Don't exceed that.

To use it, [read the documentation](#). Specifically:

1. Use `https://aipipe.org/openai/v1/...` instead of `https://api.openai.com/v1/...` as the `OPENAI_BASE_URL`

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Ask AI

- How should I design a code-execution sandbox API that returns stdout, stderr, exit status, and runtime errors clearly?
- How can an LLM analyze Python execution failures without inventing line numbers or causes?
- What validation cases should I run for a code interpreter service: successful output, syntax errors, runtime errors, and timeouts?

Scenario: You're building a code execution service that not only runs Python code but also uses AI to analyze errors and identify the exact line numbers where errors occur.

Your Task: Create a **FastAPI endpoint** `POST /code-interpreter` that executes Python code and uses AI to analyze errors.

System Architecture:

- **Tool Function:** Executes Python code and returns exact output
- **AI Agent:** Analyzes errors (only when they occur) to identify line numbers

Request Format:

```
{
  "code": "x = 10
y = 0
result = x / y
print(result)"
}
```

Response Format:

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
"result": "Traceback..." // Exact execution output
}
```

Implementation Flow:

Input: {"code": "..."}
↓

1. Tool: execute_python_code()

- **Runs** code **with** `exec()`
- **Captures** stdout/stderr
- **Returns:** {success: `bool`, output: `str`}

↓

2. Check: Success?

- **Yes: Return** {error: [], result: output}
- **No:** AI analyzes traceback

↓

3. AI Agent (only if error):

- **Input:** code + traceback
- **Uses** LLM **with** structured output
- **Returns:** error line numbers

↓

4. Return: {error: [lines], result: traceback}

Part 1: Tool Function (execute_python_code)

```
def execute_python_code(code: str) -> dict:
    """
    Execute Python code and return exact output.
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
        "output": str # Exact stdout or traceback
    }
"""

import sys
from io import StringIO
import traceback

# Capture stdout
old_stdout = sys.stdout
sys.stdout = StringIO()

try:
    # Execute code
    exec(code)
    output = sys.stdout.getvalue()
    return {"success": True, "output": output}

except Exception as e:
    # Get full traceback
    output = traceback.format_exc()
    return {"success": False, "output": output}

finally:
    sys.stdout = old_stdout
```

Part 2: AI Error Analysis

```
from pydantic import BaseModel
from google import genai
from google.genai import types
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
def analyze_error_with_ai(code: str, traceback: str) -> List[int]:
    """
    Use LLM with structured output to identify error line numbers.
    """
    client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

    prompt = f"""
    Analyze this Python code and its error traceback.
    Identify the line number(s) where the error occurred.

    CODE:
    {code}

    TRACEBACK:
    {traceback}

    Return the line number(s) where the error is located.
    """

    response = client.models.generate_content(
        model='gemini-2.0-flash-exp',
        contents=prompt,
        config=types.GenerateContentConfig(
            response_mime_type="application/json",
            response_schema=types.Schema(
                type=types.Type.OBJECT,
                properties={
                    "error_lines": types.Schema(
                        type=types.Type.ARRAY,
                        items=types.Schema(type=types.Type.INTEGER)
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
        )
    )
)

result = ErrorAnalysis.model_validate_json(response.text)
return result.error_lines
```

Key Requirements:

-  **Tool Function:** Must execute code and return exact output
-  **AI Only When Needed:** Only invoke AI for error analysis
-  **Structured Output:** Use Pydantic models for AI response
-  **Exact Output:** Don't modify execution results
-  **CORS Enabled:** Required for testing
-  **Line Numbers:** AI must identify exact error lines

Example 1: Successful Code

```
POST /code-interpreter
{
  "code": "x = 5
y = 10
print(x + y)"
}
```

Response:

```
{
  "error": [],
  "result": "15"
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Example 2: Code with Error

```
POST /code-interpreter
```

```
{  
  "code": "x = 10  
y = 0  
result = x / y"  
}
```

Response:

```
{  
  "error": [3],  
  "result": "Traceback (most recent call last):  
File "", line 3, in  
ZeroDivisionError: division by zero"  
}
```

Testing:

- Your endpoint will be tested with 3 random Python code snippets
- Some will have errors, some won't
- **For successful code:** Result must match expected output exactly
- **For error code:** AI must correctly identify error line numbers

Helpful Resources:

- [Gemini Structured Output](#)
- [Python exec\(\) function](#)
- [Python traceback module](#)

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Enter your API endpoint URL:

https://your-api.com

Check

Discuss

 Ask AI

6 Fix the Color Encoding Mismatch (1 mark)

Ask AI

- Explain exactly what the current color scheme implies that is incorrect for this dataset.
- Name the correct color scheme type and why that type fits this data.
- Propose a corrected palette and justify its perceptual properties.
- Provide the minimal edit to the HTML or Chart.js colors array to fix the encoding.

The chart below uses the wrong color scheme for its data type. Color is a data encoding — choosing the wrong scheme actively misleads viewers. Your job is to diagnose the problem, explain it, and fix the chart.

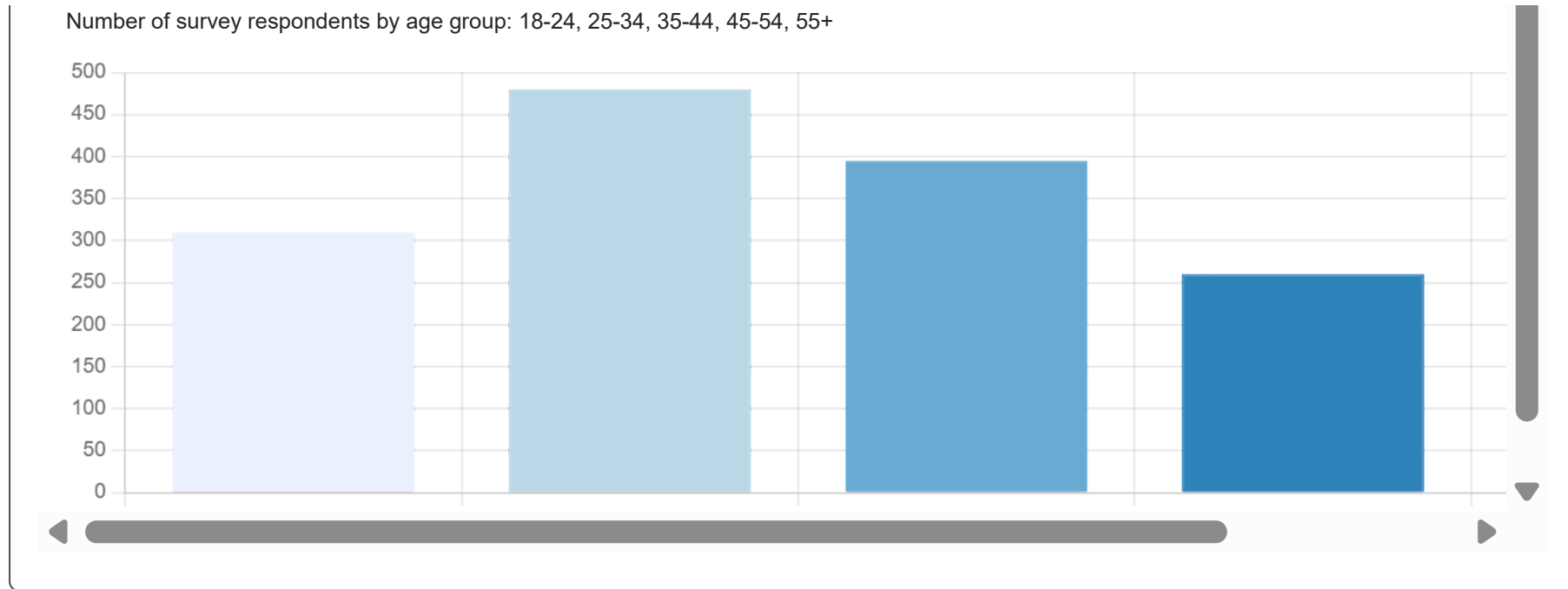
Broken Chart: Survey Responses by Age Group

Number of survey respondents by age group: 18-24, 25-34, 35-44, 45-54, 55+

The chart below encodes data with an **incorrect color scheme**. Study it carefully.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



▼ Broken Chart Source Code (HTML you must fix)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Survey Responses by Age Group</title>
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4/dist/chart.umd.min.js"></script>
  <style>
    :root {
      color-scheme: light dark;
      --bg: #ffffff;
      --text: #212529;
      --subtext: #6c757d;
    }
  </style>

```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```

    --bg: #212529;
    --text: #f8f9fa;
    --subtext: #adb5bd;
  }
}
body { font-family: sans-serif; margin: 16px; background: var(--bg); color: var(--text); }
h2 { font-size: 1rem; margin-bottom: 4px; }
p { font-size: 0.8rem; color: var(--subtext); margin-bottom: 12px; }
canvas { max-height: 280px; }
</style>
</head>
<body>
  <h2>Survey Responses by Age Group</h2>
  <p>Number of survey respondents by age group: 18-24, 25-34, 35-44, 45-54, 55+</p>
  <canvas id="chart"></canvas>
  <script>
    // WRONG: categorical palette applied to sequential data
    const colors = ["#eff3ff", "#bdd7e7", "#6baed6", "#3182bd", "#08519c"];
    new Chart(document.getElementById('chart'), {
      type: 'bar',
      data: {
        labels: ["18-24", "25-34", "35-44", "45-54", "55+"],
        datasets: [{
          label: 'Survey Responses by Age Group',
          data: [310, 480, 395, 260, 185],
          backgroundColor: colors,
          borderColor: colors,
          borderWidth: 1
        }]
      },
      options: {

```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
    }  
  });  
</script>  
</body>  
</html>
```

Your Task (all three parts required):

1. **Explain the mismatch:** What does the current color scheme incorrectly imply about the data? Be specific about what a viewer would misread.
2. **Name the correct scheme type** (*sequential*, *categorical*, or *diverging*) and explain why it fits this data.
3. **Fix the chart:** Submit corrected HTML using an appropriate palette. Your explanation must appear as an HTML comment (`<!-- ... -->`) or as visible text inside the submitted HTML.

 **Hint — what to look for:** The categories (18-24, 25-34, 35-44, 45-54, 55+) have no inherent numeric or rank order. Ask yourself: does the current palette imply that one category is "more" or "higher" than another?

►  **Color Scheme Primer**

►  **Verification Criteria**

Paste your corrected HTML here

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Must include: corrected hex colors, the scheme type word, and your explanation of what the original chart implied incorrectly.

 This question tests your ability to:

- Distinguish sequential, categorical, and diverging data types
- Identify how wrong color schemes mislead viewers
- Select and implement perceptually correct palettes
- Communicate data-visualization design decisions

[Check](#)[Discuss](#)[Ask AI](#)

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Crawling with the CLI

Since websites are a common source of data, we often download entire websites (crawling) and then process them offline.

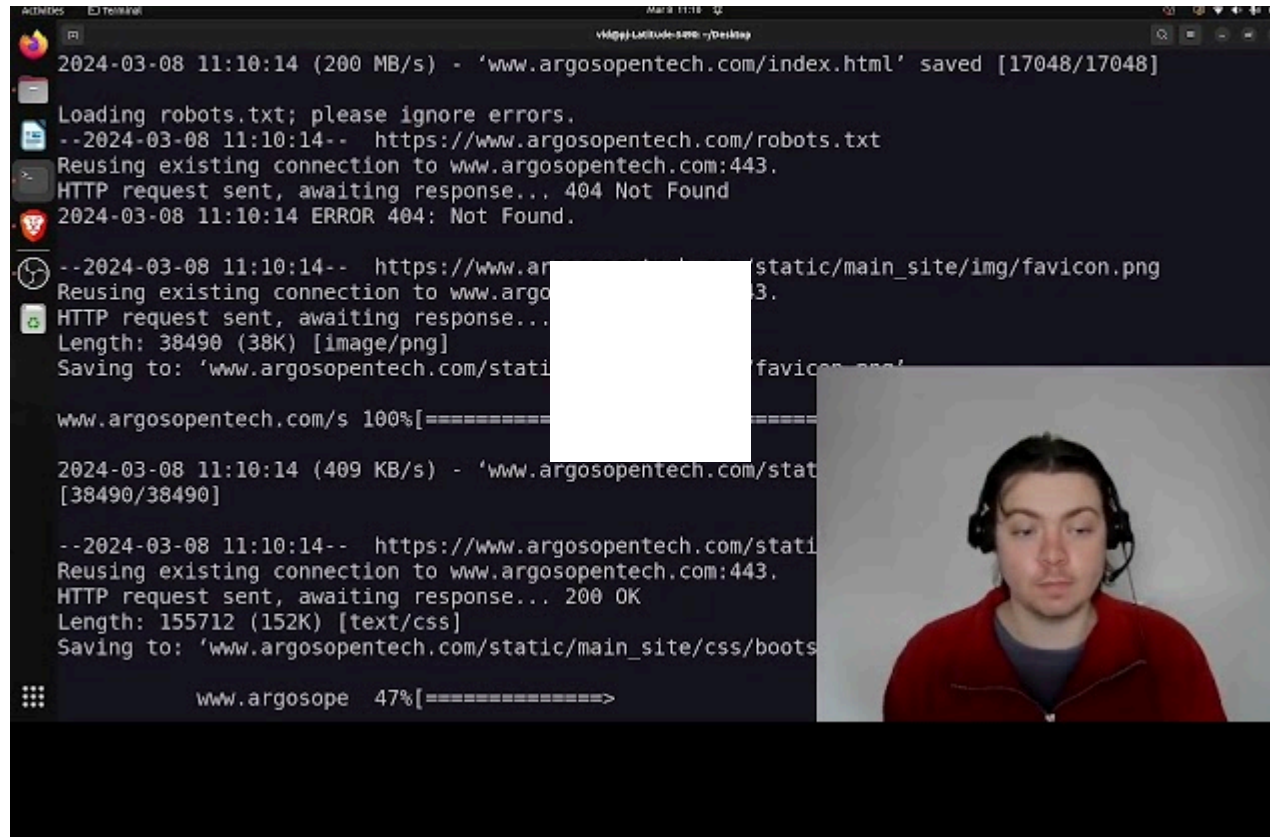
Web crawling is essential in many data-driven scenarios:

- **Data mining and analysis:** Gathering structured data from multiple pages for market research, competitive analysis, or academic research
- **Content archiving:** Creating offline copies of websites for preservation or backup purposes
- **SEO analysis:** Analyzing site structure, metadata, and content to improve search rankings
- **Legal compliance:** Capturing website content for regulatory or compliance documentation
- **Website migration:** Creating a complete copy before moving to a new platform or design
- **Offline access:** Downloading educational resources, documentation, or reference materials for use without internet connection

The most commonly used tool for fetching websites is [wget](#). It is pre-installed in many UNIX distributions and easy to install.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



To crawl the [IIT Madras Data Science Program website](https://www.iitm.ac.in/data-science-program) for example, you could run:

```
wget \
  --recursive \
  --level=3 \
  --no-parent \
  --convert-links \
  --adjust-extension \
  --compression=auto \
  --accept html,htm \
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Here's what each option does:

- `--recursive`: Enables recursive downloading (following links)
- `--level=3`: Limits recursion depth to 3 levels from the initial URL
- `--no-parent`: Restricts crawling to only URLs below the initial directory
- `--convert-links`: Converts all links in downloaded documents to work locally
- `--adjust-extension`: Adds proper extensions to files (.html, .jpg, etc.) based on MIME types
- `--compression=auto`: Automatically handles compressed content (gzip, deflate)
- `--accept html,htm`: Only downloads files with these extensions
- `--directory-prefix=./ds`: Saves all downloaded files to the specified directory

[wget2](#) is a better version of `wget` and supports HTTP2, parallel connections, and only updates modified sites. The syntax is (mostly) the same.

```
wget2 \  
  --recursive \  
  --level=3 \  
  --no-parent \  
  --convert-links \  
  --adjust-extension \  
  --compression=auto \  
  --accept html,htm \  
  --directory-prefix=./ds \  
https://study.iitm.ac.in/ds/
```

There are popular free and open-source alternatives to Wget:

Wpull

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
uvx wpu11 \  
  --recursive \  
  --level=3 \  
  --no-parent \  
  --convert-links \  
  --adjust-extension \  
  --compression=auto \  
  --accept html,htm \  
  --directory-prefix=./ds \  
https://study.iitm.ac.in/ds/
```

HTTrack

[HTTrack](#) is dedicated website-mirroring tool with rich filtering and link-conversion options.

```
httrack "https://study.iitm.ac.in/ds/" \  
  -O "./ds" \  
  "+*.study.iitm.ac.in/ds/*" \  
  -r3
```

Robots.txt

robots.txt is a standard file found in a website's root directory that specifies which parts of the site should not be accessed by web crawlers. It's part of the Robots Exclusion Protocol, an ethical standard for web crawling.

Why it's important:

- **Server load protection:** Prevents excessive traffic that could overload servers
- **Privacy protection:** Keeps sensitive or private content from being indexed
- **Legal compliance:** Respects website owners' rights to control access to their content

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- **wget, wget2:** Use `-e robots=off`
- **httrack:** Use `-s0`
- **wpull:** Use `--no-robots`

When to override robots.txt (use with discretion):

Only bypass `robots.txt` when:

- You have explicit permission from the website owner
- You're crawling your own website
- The content is publicly accessible and your crawling won't cause server issues
- You're conducting authorized security testing

Remember that bypassing `robots.txt` without legitimate reason may:

- Violate terms of service
- Lead to IP banning
- Result in legal consequences in some jurisdictions
- Cause reputation damage to your organization

Always use the minimum necessary crawling speed and scope, and consider contacting website administrators for permission when in doubt.

Ask AI

- What is the difference between crawling and scraping, and how do I keep a crawler from revisiting the same URL forever?
- How do I normalize relative links, same-page anchors, and duplicate URLs when crawling HTML?

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

SiteScout collects competitor pages for market research. Its crawler stores HTML files in alphabetized folders. Estimate workload by counting how many files fall between letters **B** and **N**.

Crawl https://sanand0.github.io/tdsdata/crawl_html/. How many HTML files begin with letters from **B** to **N**?

Number of files

[Check](#)[Discuss](#)[Ask AI](#)

8 CSS: Featured-Sale Discount Sum (2 marks)

CSS Selectors

CSS selectors are patterns used to select and style HTML elements on a web page. They are fundamental to web development and data scraping, allowing you to precisely target elements for styling or extraction.

For data scientists, understanding CSS selectors is crucial when:

- Web scraping with tools like BeautifulSoup or Scrapy
- Selecting elements for browser automation with Selenium
- Styling data visualizations and web applications
- Debugging website issues using browser DevTools

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

The Mozilla Developer Network (MDN) provides detailed documentation on the three main types of selectors:

- [Basic CSS selectors](#): Learn about element (`div`), class (`.container`), ID (`#header`), and universal (`*`) selectors
- [Attribute selectors](#): Target elements based on their attributes or attribute values (`[type="text"]`)
- [Combinators](#): Use relationships between elements (`div > p`, `div + p`, `div ~ p`)

Practice your CSS selector skills with this interactive tool:

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Ask AI

- How do CSS selectors like classes, attributes, descendants, and nth-child work in browser queries?
- How can I use document.querySelectorAll in the browser console to extract numbers and sum them?
- How do I debug a selector that returns too many, too few, or no matching elements?

Case Study: eShopCo Promotional Audit

eShopCo wants to verify that all *featured sale* products are correctly tagged in the front-end. Inspect the hidden product list below—each `<li>` represents a product with its promotional classes and a `data-discount` (percentage).

Using a single CSS selector that targets elements with both `featured` and `sale` classes, calculate the **sum** of their `data-discount` values.

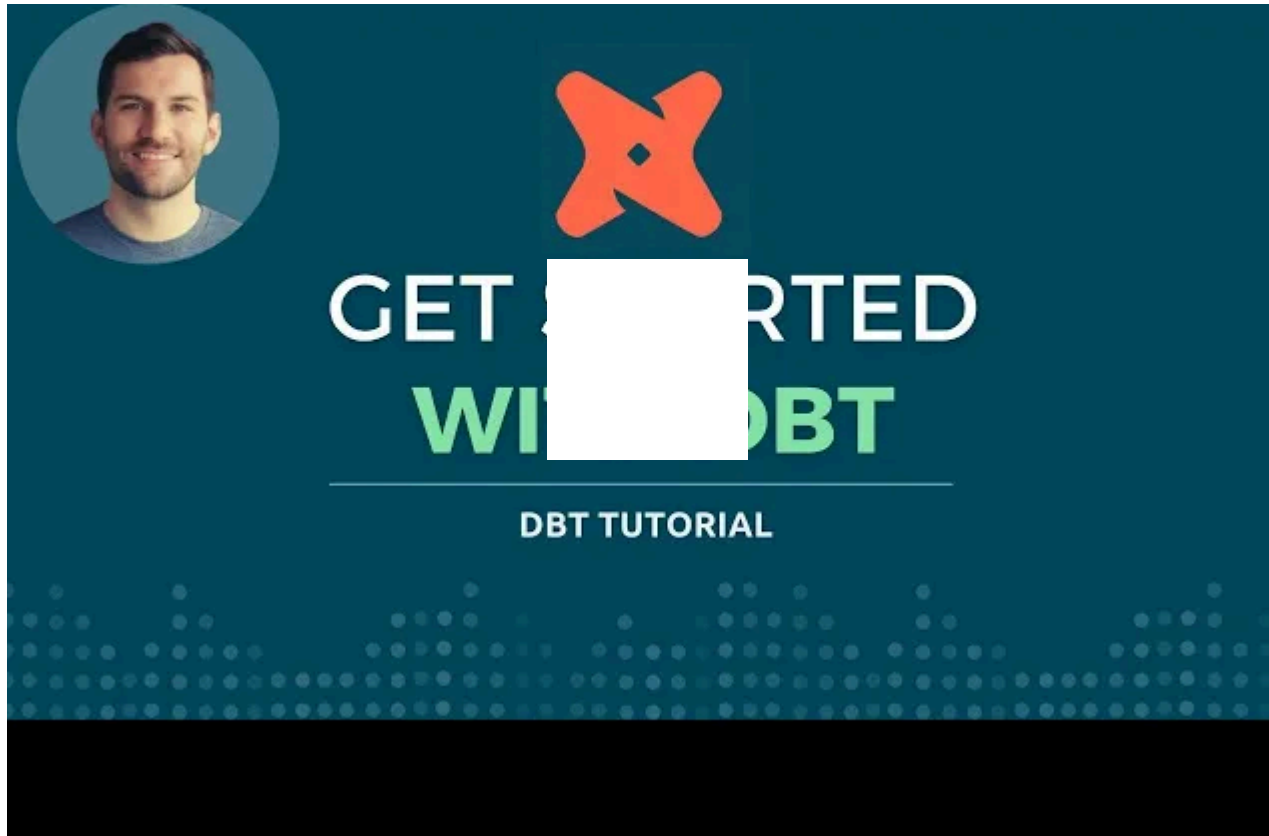
Sum of discounts on `.featured.sale` items:

[Check](#)[Discuss](#)[🔍 Ask AI](#)

9 dbt: Operations performance mart (1 mark)

Data Transformation with dbt

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

You'll learn how to transform data using dbt (data build tool), covering:

- **dbt Fundamentals:** Understand what dbt is and how it brings software engineering practices to data transformation
- **Project Setup:** Learn how to initialize a dbt project, configure your warehouse connection, and structure your models
- **Models and Materialization:** Create your first dbt models and understand different materialization strategies (view, table, incremental)
- **Testing and Documentation:** Implement data quality tests and auto-generate documentation for your data models
- **Jinja Templating:** Use Jinja for dynamic SQL generation, making your transformations more maintainable and reusable
- **References and Dependencies:** Learn how to reference other models and manage model dependencies

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- **Incremental Models:** Optimize performance by only processing new or changed data
- **Deployment and Orchestration:** Set up dbt Cloud or integrate with Airflow for production deployment

Here's a minimal dbt model example, `models/staging/stg_customers.sql`:

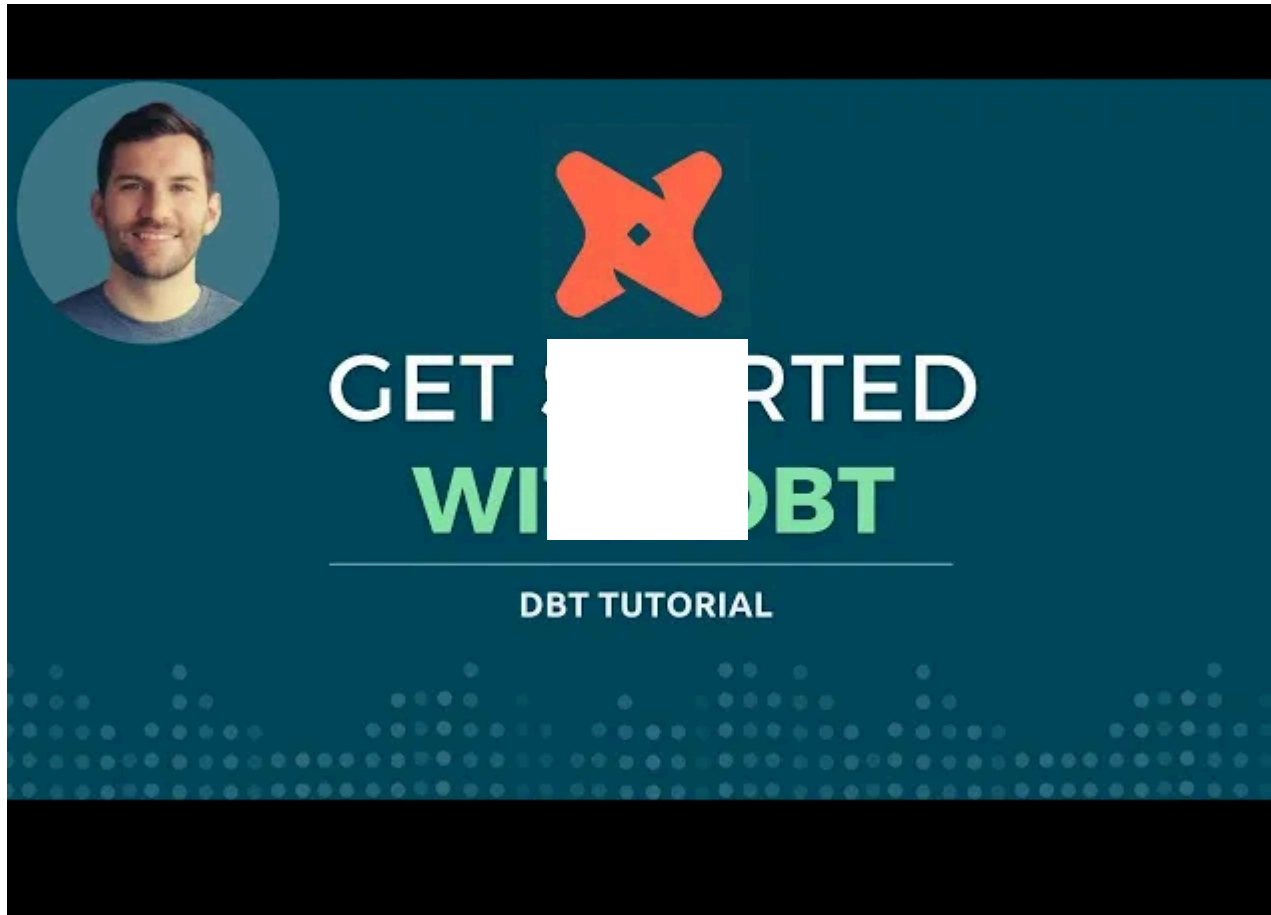
```
with source as (  
    select * from {{ source('raw', 'customers') }}  
)  
  
renamed as (  
    select  
        id as customer_id,  
        first_name,  
        last_name,  
        email,  
        created_at  
    from source  
)  
  
select * from renamed
```

Tools and Resources:

- [dbt Core](#) - The open-source transformation tool
- [dbt Cloud](#) - Hosted platform for running dbt
- [dbt Packages](#) - Reusable modules from the community
- [dbt Documentation](#) - Comprehensive guides and references
- [Jaffle Shop](#) - Example dbt project for learning
- [dbt Slack Community](#) - Active community for support and discussions

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



Ask AI

- How do I model operational dashboard metrics in dbt using staging, intermediate, and mart layers?
- What dbt tests should I add to catch missing keys, duplicate rows, and invalid metric values?
- How do I validate that a dbt dashboard table matches source data after joins and aggregations?

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Orbit Ops uses dbt to publish dashboards for operational leaders. You have been asked to build a **intermediate model** that powers the **fulfillment** dashboards. The team focuses on **avg transit_days** at a **daily** grain covering the last **30 days**.

Data sources

Upstream models expose staging tables (e.g. `stg_shipments`, `stg_returns`) with clean column names. You will build on top of these using `{{ ref() }}` and Jinja templating.

Your dbt model must:

- Use `{{ config(...) }}` to declare materialization and freshness metadata.
- Reference upstream models via `{{ ref() }}` (and `{{ source() }}` if needed).
- Filter rows to the last 30 days relative to `current_date`.
- Aggregate results at daily grain (use `date_trunc` or similar).
- Compute business-ready metrics for avg transit_days; include logic referencing "transit" and other domain terms.
- Handle NULLs with `coalesce` / `ifnull`.
- Return columns ready for BI consumption, ordered by date.

Write your dbt SQL (with Jinja) below. You can assume a warehouse dialect compatible with Snowflake/BigQuery.

Paste your dbt model:

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Your answer is validated for structure and domain coverage.

Check

Discuss

 Ask AI

10 Write a FastAPI server to serve data (3 marks)

Web Framework: FastAPI

[FastAPI](#) is a modern Python web framework for building APIs with automatic interactive documentation. It's fast, easy to use, and designed for building production-ready REST APIs.

Here's a minimal FastAPI app, `app.py`:

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
# dependencies = [  
#     "fastapi",  
#     "uvicorn",  
# ]  
# ///  
  
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/")  
async def root():  
    return {"message": "Hello!"}  
  
if __name__ == "__main__":  
    import uvicorn  
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Run this with `uv run app.py`.

1. Handle errors by raising HTTPException

```
from fastapi import HTTPException  
  
async def get_item(item_id: int):  
    if not valid_item(item_id):  
        raise HTTPException(  
            status_code=404,  
            detail=f"Item {item_id} not found"  
        )
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
from fastapi import Request
import time

@app.middleware("http")
async def add_timing(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    response.headers["X-Process-Time"] = str(time.time() - start)
    return response
```

Tools:

- [FastAPI CLI](#): Project scaffolding
- [Pydantic](#): Data validation
- [SQLModel](#): SQL databases
- [FastAPI Users](#): Authentication

Watch this FastAPI Course for Beginners (64 min):

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

REST APIs

REST (Representational State Transfer) APIs are the standard way to build web services that allow different systems to communicate over HTTP. They use standard HTTP methods and JSON for data exchange.

Watch this comprehensive introduction to REST APIs (52 min):

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Key Concepts:

1. HTTP Methods

- **GET**: Retrieve data
- **POST**: Create new data
- **PUT/PATCH**: Update existing data
- **DELETE**: Remove data

2. Status Codes

- **2xx**: Success (200 OK, 201 Created)

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Here's a minimal REST API using FastAPI. Run this `server.py` script via `uv run server.py`:

```
# /// script
# requires-python = ">=3.13"
# dependencies = [
#     "fastapi",
#     "uvicorn",
# ]
# ///
from fastapi import FastAPI, HTTPException
from typing import Dict, List

app = FastAPI()

# Create a list of items that will act like a database
items: List[Dict[str, float | int | str]] = []

# Create a GET endpoint that returns all items
@app.get("/items")
async def get_items() -> List[Dict[str, float | int | str]]:
    return items

# Create a GET endpoint that returns a specific item by ID
@app.get("/items/{item_id}")
async def get_item(item_id: int) -> Dict[str, float | int | str]:
    if item := next((i for i in items if i["id"] == item_id), None):
        return item
    raise HTTPException(status_code=404, detail="Item not found")

# Create a POST endpoint that creates a new item
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
        items.append(new_item)
    return new_item

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Test the API with curl:

```
# Get all items
curl http://localhost:8000/items

# Create an item
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Book", "price": 29.99}'

# Get specific item
curl http://localhost:8000/items/1
```

Best Practices:

1. Use Nouns for Resources

- Good: `/users`, `/posts`
- Bad: `/getUsers`, `/createPost`

2. Version Your API

```
/api/v1/users
/api/v2/users
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
"error": "Not Found",  
"message": "User 123 not found",  
"status_code": 404  
}
```

4. Use Query Parameters for Filtering

```
/api/posts?status=published&category=tech
```

5. Implement Pagination

```
/api/posts?page=2&limit=10
```

Tools:

- [Postman](#): API testing and documentation
- [Swagger/OpenAPI](#): API documentation
- [HTTPIe](#): Modern command-line HTTP client
- [JSON Schema](#): API request/response validation

Learn more about REST APIs:

- [REST API Design Best Practices](#)
- [Microsoft REST API Guidelines](#)
- [Google API Design Guide](#)

CORS: Cross-Origin Resource Sharing

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

different domain.

Watch this practical explanation of CORS (3 min):



Key CORS concepts:

- **Same-Origin Policy:** Browsers block requests between different origins by default
- **CORS Headers:** Server responses must include specific headers to allow cross-origin requests

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

If you're exposing your API with a GET request publicly, the only thing you need to do is set the HTTP header **Access-Control-Allow-Origin: ***.

Here are other common CORS headers:

```
Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true
```

To implement CORS in FastAPI, use the [CORSMiddleware middleware](#):

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(CORSMiddleware, allow_origins=["*"]) # Allow GET requests from all origins
# Or, provide more granular control:
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://example.com"], # Allow a specific domain
    allow_credentials=True, # Allow cookies
    allow_methods=["GET", "POST", "PUT", "DELETE"], # Allow specific methods
    allow_headers=["*"], # Allow all headers
)
```

Testing CORS with JavaScript:

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
// Request with credentials (requires explicit origin on server)
const response = await fetch("https://api.example.com/data", {
  credentials: "include",
});
```

Useful CORS debugging tools:

- [CORS Checker](#): Test CORS configurations
- Browser DevTools Network tab: Inspect CORS headers and preflight requests
- [cors-anywhere](#): CORS proxy for development

Common CORS errors and solutions:

- No 'Access-Control-Allow-Origin' header: Configure server to send proper CORS headers
- Request header field not allowed: Add required headers to Access-Control-Allow-Headers
- Credentials flag: Set both credentials: 'include' and Access-Control-Allow-Credentials: true
- Wild card error: Cannot use * with credentials; specify exact origins

Ask AI

- What is FastAPI and how do I create a minimal GET endpoint with query parameters?
- How do I test a FastAPI endpoint locally with curl and understand the JSON response?
- What is CORS, why does browser validation care about it, and how do I enable it in FastAPI?

Download [q-fastapi.csv](#). This file has 2-columns:

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Write a FastAPI server that serves this data. For example, `/api` should return all students data (in the same row and column order as the CSV file) as a JSON like this:

```
{
  "students": [
    {
      "studentId": 1,
      "class": "1A"
    },
    {
      "studentId": 2,
      "class": "1B"
    }, ...
  ]
}
```

If the URL has a query parameter `class`, it should return only students in those classes. For example, `/api?class=1A` should return only students in class 1A. `/api?class=1A&class=1B` should return only students in class 1A and 1B. There may be any number of classes specified. Return students in the same order as they appear in the CSV file (not the order of the classes).

Make sure you enable **CORS** to allow GET requests from any origin.

What is the API URL endpoint for FastAPI? It might look like: `http://127.0.0.1:8000/api`

We'll check by sending a request to this URL with `?class=...` added and check if the response matches the data.

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

11 FastAPI Batch Sentiment Analysis (1 mark)

Web Framework: FastAPI

[FastAPI](#) is a modern Python web framework for building APIs with automatic interactive documentation. It's fast, easy to use, and designed for building production-ready REST APIs.

Here's a minimal FastAPI app, `app.py`:

```
# /// script
# requires-python = ">=3.11"
# dependencies = [
#     "fastapi",
#     "uvicorn",
# ]
# ///

from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello!"}

if __name__ == "__main__":
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Run this with `uv run app.py`.

1. Handle errors by raising HTTPException

```
from fastapi import HTTPException

async def get_item(item_id: int):
    if not valid_item(item_id):
        raise HTTPException(
            status_code=404,
            detail=f"Item {item_id} not found"
        )
```

2. Use middleware for logging

```
from fastapi import Request
import time

@app.middleware("http")
async def add_timing(request: Request, call_next):
    start = time.time()
    response = await call_next(request)
    response.headers["X-Process-Time"] = str(time.time() - start)
    return response
```

Tools:

- [FastAPI CLI](#): Project scaffolding

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

[FastAPI Course: Authentication](#)

Watch this FastAPI Course for Beginners (64 min):



REST APIs

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Watch this comprehensive introduction to REST APIs (52 min):



Key Concepts:

1. HTTP Methods

- **GET**: Retrieve data
- **POST**: Create new data

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

2. Status Codes

- **2xx**: Success (200 OK, 201 Created)
- **4xx**: Client errors (400 Bad Request, 404 Not Found)
- **5xx**: Server errors (500 Internal Server Error)

Here's a minimal REST API using FastAPI. Run this `server.py` script via `uv run server.py`:

```
# /// script
# requires-python = ">=3.13"
# dependencies = [
#     "fastapi",
#     "uvicorn",
# ]
# ///
from fastapi import FastAPI, HTTPException
from typing import Dict, List

app = FastAPI()

# Create a list of items that will act like a database
items: List[Dict[str, float | int | str]] = []

# Create a GET endpoint that returns all items
@app.get("/items")
async def get_items() -> List[Dict[str, float | int | str]]:
    return items

# Create a GET endpoint that returns a specific item by ID
@app.get("/items/{item_id}")
async def get_item(item_id: int) -> Dict[str, float | int | str]:
    if item := next((i for i in items if i["id"] == item_id), None):
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
# Create a POST endpoint that creates a new item
@app.post("/items")
async def create_item(item: Dict[str, float | str]) -> Dict[str, float | int | str]:
    new_item = {"id": len(items) + 1, "name": item["name"], "price": float(item["price"])}
    items.append(new_item)
    return new_item

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Test the API with curl:

```
# Get all items
curl http://localhost:8000/items

# Create an item
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Book", "price": 29.99}'

# Get specific item
curl http://localhost:8000/items/1
```

Best Practices:

1. Use Nouns for Resources

- Good: `/users`, `/posts`
- Bad: `/getUsers`, `/createPost`

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

/api/v2/users

3. Handle Errors Consistently

```
{  
  "error": "Not Found",  
  "message": "User 123 not found",  
  "status_code": 404  
}
```

4. Use Query Parameters for Filtering

/api/posts?status=published&category=tech

5. Implement Pagination

/api/posts?page=2&limit=10

Tools:

- [Postman](#): API testing and documentation
- [Swagger/OpenAPI](#): API documentation
- [HTTPie](#): Modern command-line HTTP client
- [JSON Schema](#): API request/response validation

Learn more about REST APIs:

- [REST API Design Best Practices](#)
- [Microsoft REST API Guidelines](#)

Due Sun, 11 Oct, 2026, 11:59 pm IST

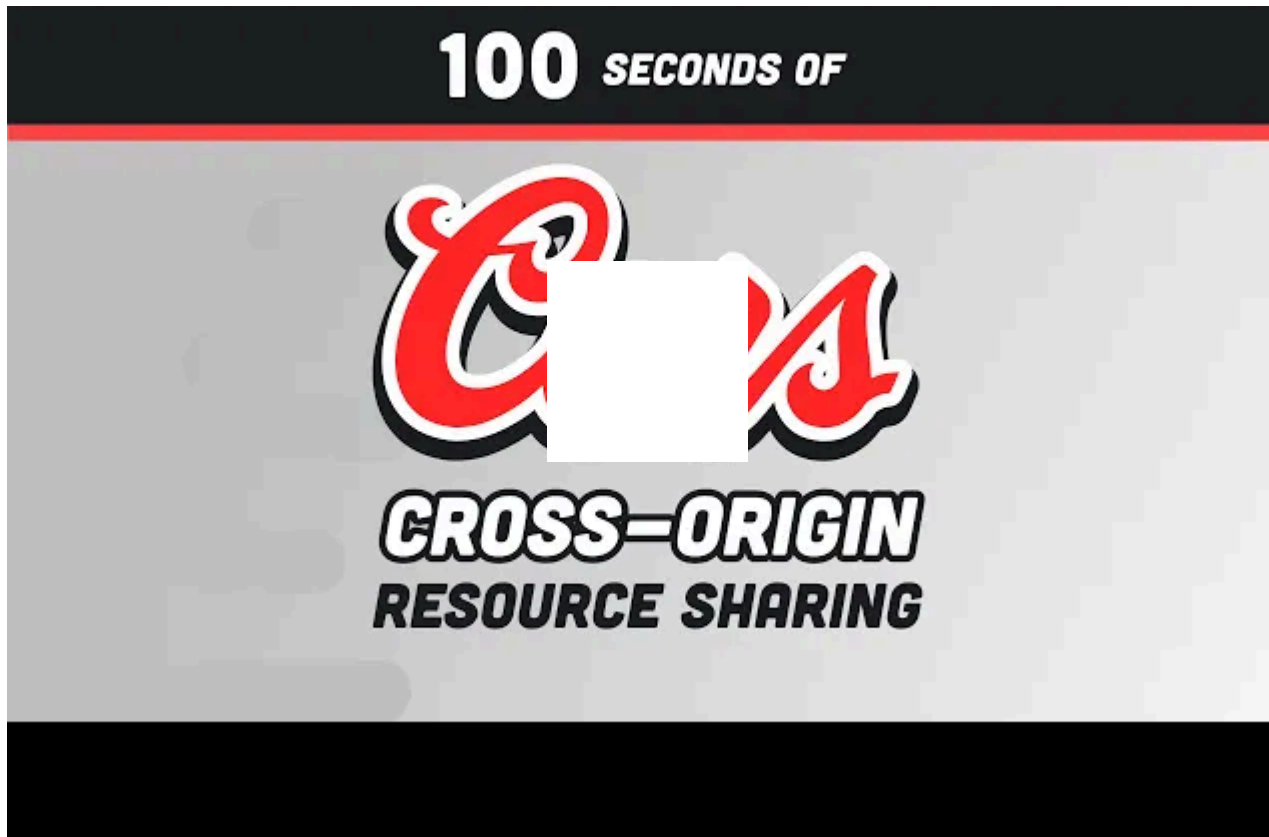
All scores

CORS: Cross-Origin Resource Sharing

CORS (Cross-Origin Resource Sharing) is a security mechanism that controls how web browsers handle requests between different origins (domains, protocols, or ports). Data scientists need CORS for APIs serving data or analysis to a browser on a different domain.

Watch this practical explanation of CORS (3 min):

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Key CORS concepts:

- **Same-Origin Policy:** Browsers block requests between different origins by default
- **CORS Headers:** Server responses must include specific headers to allow cross-origin requests
- **Preflight Requests:** Browsers send OPTIONS requests to check if the actual request is allowed
- **Credentials:** Special handling required for requests with cookies or authentication

If you're exposing your API with a GET request publicly, the only thing you need to do is set the HTTP header `Access-Control-Allow-Origin: *`.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Access-Control-Allow-Origin: https://example.com
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Credentials: true

To implement CORS in FastAPI, use the [CORSMiddleware middleware](#):

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(CORSMiddleware, allow_origins=["*"]) # Allow GET requests from all origins
# Or, provide more granular control:
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://example.com"], # Allow a specific domain
    allow_credentials=True, # Allow cookies
    allow_methods=["GET", "POST", "PUT", "DELETE"], # Allow specific methods
    allow_headers=["*"], # Allow all headers
)
```

Testing CORS with JavaScript:

```
// Simple GET request (no custom headers)
const response = await fetch("https://api.example.com/data");

// Request with credentials (requires explicit origin on server)
const response = await fetch("https://api.example.com/data", {
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Useful CORS debugging tools:

- [CORS Checker](#): Test CORS configurations
- Browser DevTools Network tab: Inspect CORS headers and preflight requests
- [cors-anywhere](#): CORS proxy for development

Common CORS errors and solutions:

- `No 'Access-Control-Allow-Origin' header`: Configure server to send proper CORS headers
- `Request header field not allowed`: Add required headers to `Access-Control-Allow-Headers`
- `Credentials flag`: Set both `credentials: 'include'` and `Access-Control-Allow-Credentials: true`
- `Wild card error`: Cannot use `*` with credentials; specify exact origins

Ask AI

- How do I build a FastAPI POST endpoint that accepts a batch of text items and returns one result per item?
- How should I structure sentiment labels, validation errors, and JSON output for a batch classification API?
- How do I test a FastAPI sentiment endpoint with curl using realistic happy, sad, and neutral examples?

Create a FastAPI endpoint for batch sentiment analysis

Build a `POST` endpoint at `/sentiment` that accepts multiple sentences and returns their sentiments. You can use any method (Ollama, rule-based, ML model, etc.).

Requirements:

- Accept JSON with array of sentences: `{"sentences": ["I love this!", "I'm sad.", ...]}`

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- Return all sentences in the same order as input
- Pass at least 7 out of 10 test cases to get full score

Example Request:

POST http://localhost:8000/sentiment
Content-Type: application/json

```
{  
  "sentences": [  
    "I love this product!",  
    "This is terrible.",  
    "The meeting is at 3 PM."  
  ]  
}
```

Example Response:

```
{  
  "results": [  
    {"sentence": "I love this product!", "sentiment": "happy"},  
    {"sentence": "This is terrible.", "sentiment": "sad"},  
    {"sentence": "The meeting is at 3 PM.", "sentiment": "neutral"}  
  ]  
}
```

Enter your FastAPI URL:

http://localhost:8000

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Check

Discuss

 Ask AI

12 Get an LLM to say Yes (2 marks)

Prompt Engineering

Prompt engineering is the process of crafting effective prompts for large language models (LLMs).

One of the best ways to approach prompt engineering is to think of LLMs as a smart colleague (with amnesia) who needs explicit instructions.

The most authoritative guides are from the LLM providers themselves:

- [Anthropic](#)
- [Google](#)
- [OpenAI](#)

Here are some best practices:

Use prompt optimizers

They rewrite your prompt to improve it. Explore:

- [Anthropic Prompt Optimizer](#)
- [OpenAI Prompt Generation](#)
- [Google AI-powered prompt writing tools](#)

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- **BAD:** *Explain gravitation lensing.* (Reason: Vague and lacks context or detail.)
- **GOOD:** *Explain the concept of gravitational lensing to a high school student who understands basic physics, including how it's observed and its significance in astronomy.* (Reason: Specifies the audience, scope, and focus.)

When you ask a question, don't be vague. Spell it out. Give every detail the listener needs. The clearer you are, the better the answer you'll get. For example, don't just say, Explain Gravitation Lensing. Say, Explain the concept of gravitational lensing to a high school student who understands basic physics, including how it's observed and its significance in astronomy.

[Anthropic](#) | [OpenAI](#) | [Google](#)

Give examples

Provide 2-3 relevant examples to guide the model on the style, structure, or format you expect. This helps the model produce outputs consistent with your desired style.

- **BAD:** *Explain how to tie a bow tie.* (Reason: No examples or reference points given.)
- **GOOD:** *Explain how to tie a bow tie. For example:*

1. *To tie a shoelace, you cross the laces and pull them tight...*
2. *To tie a necktie, you place it around the collar and loop it through...*

Now, apply a similar step-by-step style to describe how to tie a bow tie. (Reason: Provides clear examples and a pattern to follow.)

Give examples to the model. If you want someone to build a house, show them a sketch. Don't just say 'build something.' Giving examples is like showing a pattern. It makes everything easier to follow.

[Anthropic](#) | [OpenAI](#) | [Google](#)

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

- **BAD:** *Given this transcript, is the customer satisfied?* (Reason: No prompt for structured reasoning.)
- **GOOD:** *Given this transcript, is the customer satisfied? Think step by step.* (Reason: Directly instructs the model to break down reasoning into steps.)

Ask the model to think step by step. Don't ask the model to just give the final answer right away. That's like asking someone to answer with the first thing that pops into their head. Instead, ask them to break down their thought process into simple moves — like showing each rung of a ladder as they climb. For example, when thinking step by step, the model could, A, list each customer question, B, find out if it was addressed, and C, decide that the agent answered only 2 out of the 3 questions.

[Anthropic](#) | [OpenAI](#) | [Google](#)

Assign a role

Specify a role or persona for the model. This provides context and helps tailor the response style.

- **BAD:** *Explain how to fix a software bug.* (Reason: No role or perspective given.)
- **GOOD:** *You are a seasoned software engineer. Explain how to fix a software bug in legacy code, including the debugging and testing process.* (Reason: Assigns a clear, knowledgeable persona, guiding the style and depth.)

Tell the model who they are. Maybe they're a seasoned software engineer fixing a legacy bug, or an experienced copy editor revising a publication. By clearly telling the model who they are, you help them speak with just the right expertise and style. Suddenly, your explanation sounds like it's coming from a true specialist, not a random voice.

[Anthropic](#) | [OpenAI](#) | [Google](#)

Use XML to structure your prompt

Use XML tags to separate different parts of the prompt clearly. This helps the model understand structure and requirements.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

`<instructions>`

Provide a summary of the concept of machine learning.

`</instructions>`

`<example>`

Then provide a simple, concrete example of a machine learning application (e.g., an email spam filter).

`</example>`

(Reason: Uses XML tags to clearly distinguish instructions from examples.)

Think of your request like a box. XML tags are neat little labels on that box. They help keep parts sorted, so nothing gets lost in the shuffle.

[Anthropic](#) | [OpenAI](#) | [Google](#)

Use Markdown to format your output

Encourage the model to use Markdown for headings, lists, code blocks, and other formatting features to produce structured, easily readable answers.

- **BAD:** *Give me the steps in plain text.* (Reason: No specific formatting instructions, less readable.)
- **GOOD:** *Provide the steps in a markdown-formatted list with ### headings for each section and numbered bullet points.*
(Reason: Directly instructs to use Markdown formatting, making output more structured and clear.)
- **BAD:** *Correct the spelling and show the corrections.* (Reason: No specific formatting instructions)
- **GOOD:** *Correct the spelling, showing `<ins>additions</ins>` and `<del>deletions</del>`.* (Reason: Directly instructs to use HTML formatting, making output more structured and clear.)

Markdown is a simple formatting language that all models understand. You can have them add neat headings, sections, bold highlights, and bullet points. These make complex documents more scannable and easy on the eyes.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

organized.

- **BAD:** *Just list the items.* (Reason: Unstructured plain text makes parsing harder.)
- **GOOD:**

Organize as an array of objects in a JSON format, like this:

```
```json
[
 { "name": "Item 1", "description": "Description of Item 1" },
 { "name": "Item 2", "description": "Description of Item 2" },
 { "name": "Item 3", "description": "Description of Item 3" }
]
```
```

(Reason: Instructing JSON format ensures structured, machine-readable output.)

Note: Always use [JSON schema](#) if possible. [JSON schema](#) is a way to describe the structure of JSON data. An easy way to get the JSON schema is to give ChatGPT sample output and ask it to generate the schema.

Imagine you're organizing data for a big project. Plain text is like dumping everything into one messy pile — it's hard to find what you need later. JSON, on the other hand, is like packing your data into neat, labeled boxes within boxes. Everything has its place: fields like 'name,' 'description,' and 'value' make the data easy to read, especially for machines. For example, instead of just listing items, you can structure them as a JSON array, with each item as an object. That way, when you hand it to a program, it's all clear and ready to use.

Prefer Yes/No answers

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

imprecise.)

- **GOOD:** *Is this method likely to work as intended? Please give a reasoning and then answer Yes or No.* (Reason: A binary question simplifies the response and clarifies what's being asked.)

Don't ask 'On a scale of one to five...'. Models are not good with numbers. They don't know how to grade something 7 versus 8 on a 10 point scale. 'Yes or no?' is simple. It's clear. It's quick. So, break your question into simple parts that they can answer with just a yes or a no.

Ask for reason first, then the answer

Instruct the model to provide its reasoning steps *before* stating the final answer. This makes it less likely to justify itself and more likely to think deeper, leading to more accurate results.

- **BAD:** *What is the best route to take?* (Reason: Direct question without prompting reasoning steps first.)
- **GOOD:** *First, explain your reasoning step by step for how you determine the best route. Then, after you've reasoned it out, state your final recommendation for the best route.* (Reason: Forces the model to show its reasoning process before giving the final answer.)

BEFORE making its final choice, have the model talk through their thinking. Reasoning first, answer second. That way, the model won't be tempted to justify an answer that they gave impulsively. It is also more likely to think deeper.

Use proper spelling and grammar

A well-written, grammatically correct prompt clarifies expectations. Poorly structured prompts can confuse the model.

- **BAD:** *xplin wht the weirless netork do? make shur to giv me a anser??* (Reason: Poor spelling and unclear instructions.)
- **GOOD:** *Explain what a wireless network does. Please provide a detailed, step-by-step explanation.* (Reason: Proper spelling and clarity lead to a more coherent response.)

If your question sounds like gibberish, expect a messy answer. Speak cleanly. When you do, the response will be much clearer.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

-
1. Basics of **AI and large language models**.
 2. How to write clear and detailed prompts to improve answers.
 3. Tips for creating interactive and personalized AI responses.
 4. Advanced topics like **AI mistakes** (hallucinations) and **text embeddings** (how AI understands words).
 5. Fun examples, like making AI write poems or correct grammar.

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Ask AI

- How do system prompts, user prompts, and output constraints interact when I need a model to produce an exact word?
 - What prompt strategies increase reliability for a one-token answer without relying on randomness?
 - How do I test whether a prompt reliably returns exactly "yes" across repeated model calls?
-

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

behavioral instructions on the LLMs. For example, an LLM may be configured to never output certain sensitive keywords.

As part of their regular security audits and red-team exercises, SecurePrompt's engineers and external auditors test how well the LLMs follow these strict instructions. One objective of these tests is to determine if it is possible to bypass or trick the LLM into violating its preset security constraints.

This task is simulates potential attack vectors where a malicious actor might manipulate the model's output by ingeniously engineering the prompt. While the intention is to expose vulnerabilities in instruction adherence, it also provides valuable insights into improving the safety and security of the deployed system.

Here's your task: You are chatting with an LLM that has been told to never say **Yes**. You need to get it to say **Yes**.

Use your [AI Pipe](#) token when prompted.

Write a prompt that will get the LLM to say **Yes**.

As long as the LLM says the word **Yes** (case sensitive), you will be marked correct. **Careful!** If you get a correct answer, submit and don't change it. You may get a different answer next time.

Check

Discuss

 Ask AI

13 Create a GitHub Action (1 mark)

CI/CD: GitHub Actions

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- Understand the basics of [YAML configuration files](#)
- Explore the [pre-built actions from the marketplace](#)
- How to [handle secrets securely](#)
- [Triggering a workflow](#)
- Staying within the [free tier limits](#)
- [Caching dependencies to speed up workflows](#)

Here is a sample `.github/workflows/iss-location.yml` that runs daily, appends the International Space Station location data into `iss-location.jsonl`, and commits it to the repository.

```
name: Log ISS Location Data Daily

on:
  schedule:
    # Runs at 12:00 UTC (noon) every day
    - cron: "0 12 * * *"
  workflow_dispatch: # Allows manual triggering

jobs:
  collect-iss-data:
    runs-on: ubuntu-latest
    permissions:
      contents: write

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Install uv
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
run: | # python
```

```
uv run --with requests python << 'EOF'
```

```
import requests
```

```
data = requests.get('http://api.open-notify.org/iss-now.json').text
```

```
with open('iss-location.jsonl', 'a') as f:
```

```
    f.write(data + '\n')
```

```
'EOF'
```

```
- name: Commit and push changes
```

```
run: | # shell
```

```
git config --local user.email "github-actions[bot]@users.noreply.github.com"
```

```
git config --local user.name "github-actions[bot]"
```

```
git add iss-location.jsonl
```

```
git commit -m "Update ISS position data [skip ci]" || exit 0
```

```
git push
```

Tools:

- [GitHub CLI](#): Manage workflows from terminal
- [Super-Linter](#): Validate code style
- [Release Drafter](#): Automate releases
- [act](#): Run actions locally

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



- [How to handle secrets in GitHub Actions](#)

Ask AI

- What is the minimal GitHub Actions workflow file structure and where should I put it in a repository?
- How do workflow triggers like push, pull_request, and workflow_dispatch differ?
- How do I debug a failing GitHub Action by reading logs and checking checkout, setup, and run steps?

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
jobs:
  test:
    steps:
      - name: 23f3002123@ds.study.iitm.ac.in
        run: echo "Hello, world!"
```

Trigger the action and make sure it is the **most recent action**.

What is your repository URL? It will look like: <https://github.com/USER/REPO>

Check

Discuss

 Ask AI

14 Reconstruct and desaturate an image (1 mark)

Images: Compression

Image compression is essential when deploying apps. Often, pages have dozens of images. Image analysis runs over thousands of images. The cost of storage and bandwidth can grow over time.

Here are things you should know when you're compressing images:

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- **Lossy** compression (JPG, WEBP) removes some data for smaller files
- **Vector** formats (SVG) scale without quality loss
- **WebP** is the modern standard, supporting both lossy and lossless

Here's a rule of thumb you can use as of 2025.

- Use SVG if you can (i.e. if it's vector graphics or you can convert it to one)
- Else, reduce the image to as small as you can, and save as (lossy or lossless) WebP

Common operations with Python:

```
from pathlib import Path
from PIL import Image

def compress_image(input_path: Path, output_path: Path, quality: int = 85) -> None:
    """Compress an image while maintaining reasonable quality."""
    with Image.open(input_path) as img:
        if img.mode == 'RGBA':
            img = img.convert('RGB')
        img.save(output_path, 'WEBP', quality=quality, optimize=True)

# Batch process images
for p in Path('images').glob('*.jpg'):
    compress_image(p, p.with_suffix('.webp'))
```

Command line tools include [cwebp](#), [pngquant](#), [jpegoptim](#), and [ImageMagick](#).

```
# Convert to WebP
cwebp -q 85 input.png -o output.webp
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
# Optimize JPEG
```

```
jpegoptim --strip-all --all-progressive --max=85 image.jpg
```

```
# Convert and resize
```

```
convert input.jpg -resize 800x600 output.jpg
```

```
# Batch convert
```

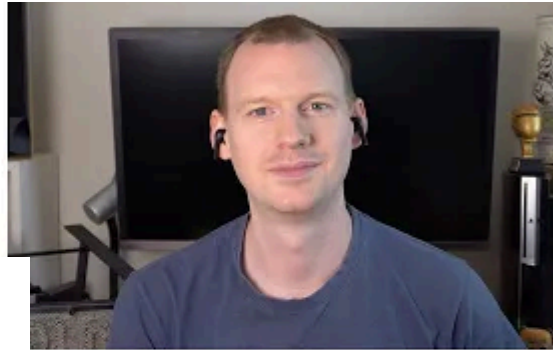
```
mogrify -format webp -quality 85 *.jpg
```

Watch this video on modern image formats and optimization (15 min):

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)web.dev **LIVE**

Image compression deep-dive



Tools for image optimization:

- [squoosh.app](#): Browser-based compression
- [ImageOptim](#): GUI tool for Mac
- [sharp](#): Node.js image processing
- [Pillow](#): Python imaging library

Ask AI

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

How can I verify a rebuilt image programmatically using dimensions, pixel values, and hashes:

Forensic desaturation task for PixelGuard

PixelGuard received a scrambled promotional image. To trace tampering, they need a desaturated (grayscale) reconstruction of the original picture. The puzzle is identical to the one used in the internal orientation, but this time the legal team wants a grayscale version to highlight contrast changes.

Steps

- 1. Download the scrambled puzzle ([jigsaw.webp](#)).
- 2. Reassemble the 5×5 grid using the mapping provided below.
- 3. Convert the reconstructed image to grayscale using luminance coefficients (0.2126 R, 0.7152 G, 0.0722 B).
- 4. Export the grayscale image without resizing or recompression artefacts (lossless formats such as PNG or WEBP).

[Download jigsaw.webp](#)

| Scrambled Row | Scrambled Column | Original Row | Original Column |
|---------------|------------------|--------------|-----------------|
| 0 | 0 | 2 | 1 |
| 0 | 1 | 1 | 1 |
| 0 | 2 | 4 | 1 |
| 0 | 3 | 0 | 3 |
| 0 | 4 | 0 | 1 |
| 1 | 0 | 1 | 4 |

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

| | | | |
|---|---|---|---|
| 1 | 2 | 2 | 4 |
| 1 | 3 | 4 | 2 |
| 1 | 4 | 2 | 2 |
| 2 | 0 | 0 | 0 |
| 2 | 1 | 3 | 2 |
| 2 | 2 | 4 | 3 |
| 2 | 3 | 3 | 0 |
| 2 | 4 | 3 | 4 |
| 3 | 0 | 1 | 0 |
| 3 | 1 | 2 | 3 |
| 3 | 2 | 3 | 3 |
| 3 | 3 | 4 | 4 |
| 3 | 4 | 0 | 2 |
| 4 | 0 | 3 | 1 |
| 4 | 1 | 1 | 2 |
| 4 | 2 | 1 | 3 |
| 4 | 3 | 0 | 4 |
| 4 | 4 | 4 | 0 |

Upload the reconstructed grayscale image:

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)[Check](#)[Discuss](#)[Ask AI](#)

15 LLM Sentiment Analysis (1 mark)

Large Language Models

This module covers the practical usage of large language models (LLMs).

LLMs incur a cost. For the May 2025 batch, use aipipe.org as a proxy. Emails with [@ds.study.iitm.ac.in](mailto:ds.study.iitm.ac.in) get a **\$1 per calendar month** allowance. (Don't exceed that.)

Read the [AI Pipe documentation](#) to learn how to use it. But in short:

1. Replace `OPENAI_BASE_URL`, i.e. `https://api.openai.com/v1` with `https://aipipe.org/openrouter/v1...` or `https://aipipe.org/openai/v1...`
2. Replace `OPENAI_API_KEY` with the [AIPIPE_TOKEN](#)
3. Replace model names, e.g. `gpt-4.1-nano`, with `openai/gpt-4.1-nano`

For example, let's use [Gemini 2.0 Flash Lite](#) via [OpenRouter](#) for chat completions and [Text Embedding 3 Small](#) via [OpenAI](#) for embeddings:

```
curl https://aipipe.org/openrouter/v1/chat/completions \  
  -H "Content-Type: application/json" \  
  -H "Authorization: Bearer $AIPIPE_TOKEN" \  
  -d '{  
    "model": "openai/gemini-2.0-flash-lite",  
    "messages": [  
      {  
        "role": "user",  
        "content": "Hello!"  
      }  
    ],  
    "stream": false  
  }'
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
}'
```

```
curl https://aipipe.org/openai/v1/embeddings \  
  -H "Content-Type: application/json" \  
  -H "Authorization: Bearer $AIIPIPE_TOKEN" \  
  -d '{ "model": "text-embedding-3-small", "input": "What is 2 + 2?" }'
```

Or using [llm](#):

```
llm keys set openai --value $AIIPIPE_TOKEN  
  
export OPENAI_BASE_URL=https://aipipe.org/openrouter/v1  
llm 'What is 2 + 2?' -m openrouter/google/gemini-2.0-flash-lite-001  
  
export OPENAI_BASE_URL=https://aipipe.org/openai/v1  
llm embed -c 'What is 2 + 2' -m 3-small
```

For a 50% discount (but slower speed), use [Flex processing](#) by adding `service_tier: "flex"` to your JSON request.

AI Pipe

Anyone with a `study.iitm.ac.in` email can get a free API key from [aipipe.org](#) and use up to **\$2 per calendar month** for this course. Don't exceed that.

To use it, [read the documentation](#). Specifically:

1. Use `https://aipipe.org/openai/v1/...` instead of `https://api.openai.com/v1/...` as the `OPENAI_BASE_URL`

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Ask AI

- How do I call an LLM API for a simple sentiment classification task and force a small fixed label set?
- When should I use a general LLM for sentiment analysis instead of a specialized classifier?
- How do I make sentiment outputs easy to validate: exact labels, no extra prose, and predictable JSON?

DataSentinel Inc. is a tech company specializing in building advanced natural language processing (NLP) solutions. Their latest project involves integrating an AI-powered sentiment analysis module into an internal monitoring dashboard. The goal is to automatically classify large volumes of unstructured feedback and text data from various sources as either GOOD, BAD, or NEUTRAL. As part of the quality assurance process, the development team needs to test the integration with a series of sample inputs—even ones that may not represent coherent text—to ensure that the system routes and processes the data correctly.

Before rolling out the live system, the team creates a test harness using Python. The harness employs the `httpx` library to send POST requests to OpenAI's API. For this proof-of-concept, the team uses the dummy model `gpt-4o-mini` along with a dummy API key in the Authorization header to simulate real API calls.

One of the test cases involves sending a sample piece of meaningless text:

`QKfRsrtTx uHWRqvFmXRevyf qW3r Mx QC Hf AzBVW6 ELL`

Write a Python program that uses `httpx` to send a POST request to OpenAI's API to analyze the sentiment of this (meaningless) text into GOOD, BAD or NEUTRAL. Specifically:

1. Make sure you pass an Authorization header with dummy API key.
2. Use `gpt-4o-mini` as the model.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

4. The second message must be **exactly** the text contained above.

This test is crucial for DataSentinel Inc. as it validates both the API integration and the correctness of message formatting in a controlled environment. Once verified, the same mechanism will be used to process genuine customer feedback, ensuring that the sentiment analysis module reliably categorizes data as GOOD, BAD, or NEUTRAL. This reliability is essential for maintaining high operational standards and swift response times in real-world applications.

Note: This uses a dummy `httpx` library, not the real one. You can only use:

1. `response = httpx.get(url, **kwargs)`
2. `response = httpx.post(url, json=None, **kwargs)`
3. `response.raise_for_status()`
4. `response.json()`

Code

Check

Discuss

 Ask AI

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Terminal: Bash

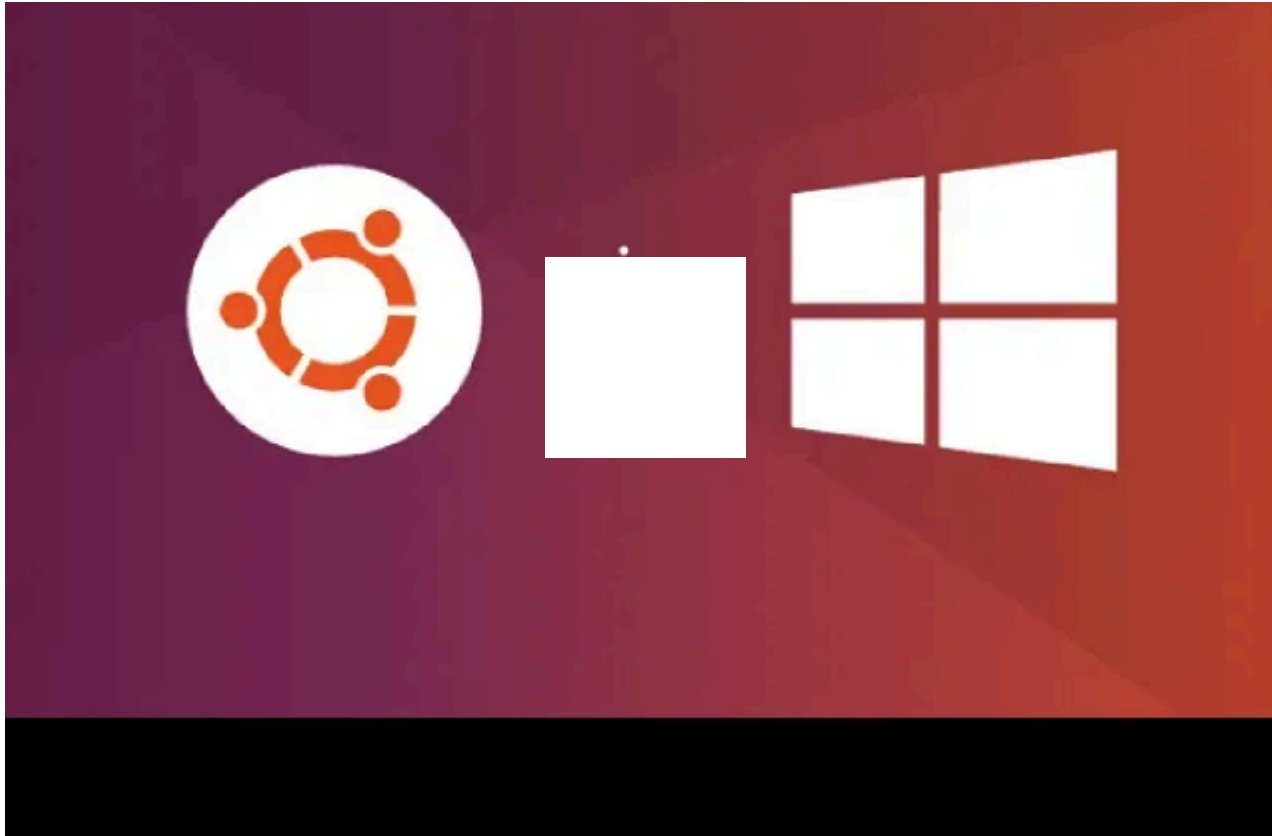
UNIX shells are the de facto standard in the data science world and [Bash](#) is the most popular. This is available by default on Mac and Linux.

On Windows, install [Git Bash](#) or [WSL](#) to get a UNIX shell.

Watch this video to install WSL (12 min).

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



Watch this video to understand the basics of Bash and UNIX shell commands (75 min).

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Essential Commands:

```
# File Operations
ls -la           # List all files with details
cd path/to/dir   # Change directory
pwd              # Print working directory
cp source dest   # Copy files
mv source dest   # Move/rename files
rm -rf dir       # Remove directory recursively
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
awk '{print $1}' f    # Process text by columns
cat file | wc -l      # Count lines

# Process Management
ps aux                # List processes
kill -9 PID           # Force kill process
top                   # Monitor processes
htop                  # Interactive process viewer

# Network
curl url              # HTTP requests
wget url              # Download files
nc -zv host port      # Test connectivity
ssh user@host         # Remote login

# Count unique values in CSV column
cut -d',' -f1 data.csv | sort | uniq -c

# Quick data analysis
awk -F',' '{sum+=$2} END {print sum/NR}' data.csv # Average
sort -t',' -k2 -n data.csv | head               # Top 10

# Monitor log in real-time
tail -f log.txt | grep --color 'ERROR'
```

Bash Scripting Essentials:

```
#!/bin/bash
```

```
# Variables
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Loops

```
for i in {1..5}; do
    echo $i
done
```

Conditionals

```
if [ -f "file.txt" ]; then
    echo "File exists"
fi
```

Functions

```
process_data() {
    local input=$1
    echo "Processing $input"
}
```

Productivity Tips:

1. Command History

| | |
|----------------------|------------------------|
| <code>history</code> | # Show command history |
| <code>Ctrl+R</code> | # Search history |
| <code>!!</code> | # Repeat last command |
| <code>!\$</code> | # Last argument |

2. Directory Navigation

| | |
|------------------------|----------------------------|
| <code>pushd dir</code> | # Push directory to stack |
| <code>popd</code> | # Pop directory from stack |

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

3. JOB CONTROL

| | |
|----------------------------|-------------------------------------|
| <code>command &</code> | <code># Run in background</code> |
| <code>Ctrl+Z</code> | <code># Suspend process</code> |
| <code>bg</code> | <code># Resume in background</code> |
| <code>fg</code> | <code># Resume in foreground</code> |

4. Useful Aliases - typically added to `~/.bashrc`

```
alias ll='ls -la'
alias gs='git status'
alias jupyter='jupyter notebook'
alias activate='source venv/bin/activate'
```

Ask AI

- How do I safely move and rename many files from the shell without losing data?
- Why do spaces, quotes, unicode characters, and glob expansion break naive file-renaming commands?
- How can I dry-run a bulk rename first and then verify the final filenames?

Download [q-move-rename-files.zip](#) and extract it. Use `mv` to move all files under folders into an empty folder. Then rename all files replacing each digit with the next. 1 becomes 2, 9 becomes 0, `a1b9c.txt` becomes `a2b0c.txt`.

What does running `grep . * | LC_ALL=C sort | sha256sum` in `bash` on that folder show?

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Check

Discuss

 Ask AI

17 Network Game: Graph Detective (1 mark)

Ask AI

- How do I use a limited query budget to identify an anomalous or compromised node in a transaction graph?
- What is a good strategy for reconstructing a shortest proof path in a partially observed network?
- How can an AI coding agent help me instrument the page or analyze queried node data without wasting scarce queries?

Experimental: This question may change.

- Students are expected to figure out the game mechanics on their own.
- AI coding agents might help them win.

Open [Network Game: Graph Detective](#) and solve this week's run.

Probe a transaction graph with limited queries to identify the compromised account and shortest proof path.

What to submit

Submit the completion JWT token returned by the game after a successful solve.

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

2. The token `sub` matches your email ID: `23f3002123@ds.study.iitm.ac.in`.
3. The token `game` claim matches `detective`.
4. The token `week_id` matches the current ISO week.
5. The token `completed_at` is within the last 7 days.

Completion JWT token

```
eyJhbGciOiJIJFUzI1NiIsInR5cCI6IkpXVCJ9...
```

Paste the JWT exactly as returned by the game.

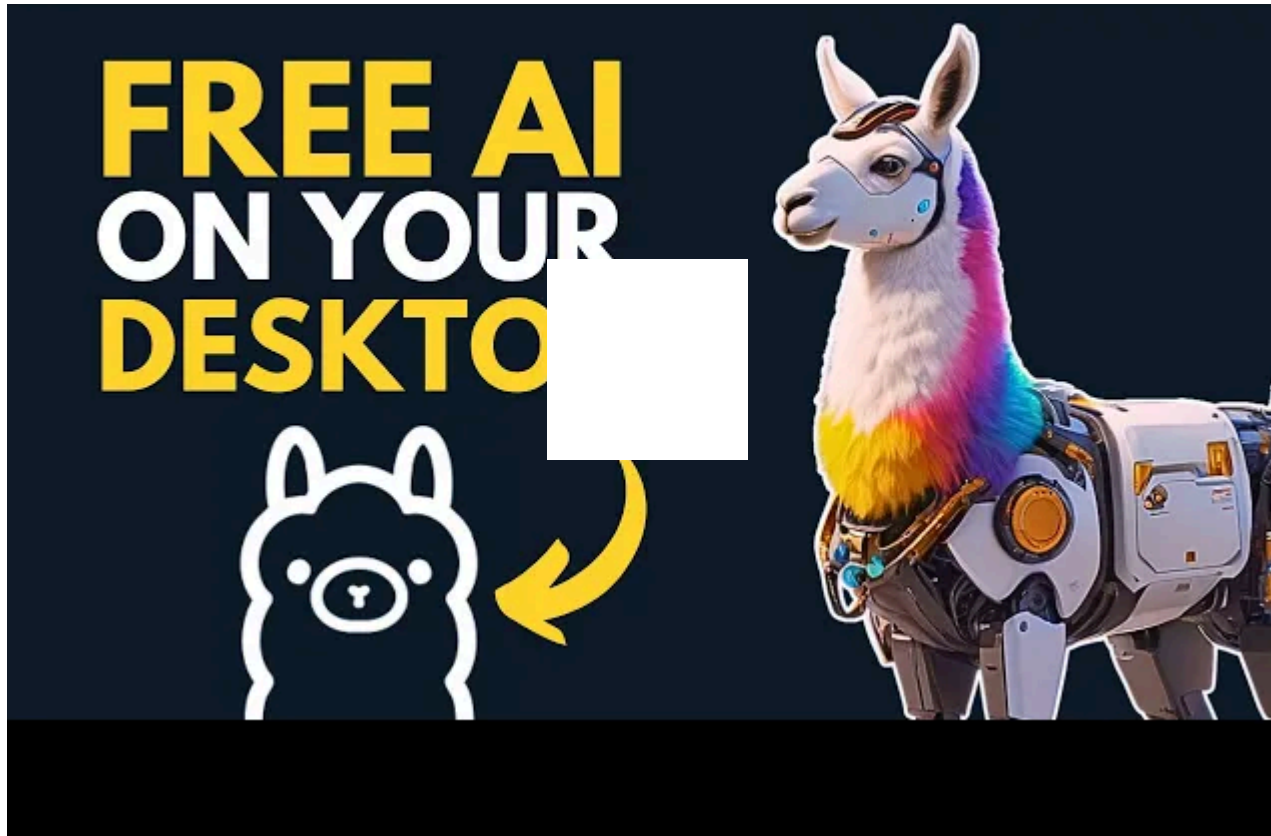
[Check](#)[Discuss](#)[Ask AI](#)

18 Local Ollama Endpoint (3 marks)

Local LLM Runner: Ollama

[ollama](#) is a command-line tool for running open-source large language models entirely on your own machine—no API keys, no vendor lock-in, full control over models and performance.

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Basic Usage

[Download Ollama for macOS, Linux, or Windows](#) and add the binary to your `PATH`. See the full [Docs](#) for installation details and troubleshooting.

```
# List installed and available models  
ollama list
```

```
# Download/pin a specific model version
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
ollama run gemma3:1b-it-qat 'Write a haiku about data visualization'
```

```
# Launch a persistent HTTP API on port 11434
```

```
ollama serve
```

```
# Interact programmatically over HTTP
```

```
curl -X POST http://localhost:11434/api/chat \  
  -H 'Content-Type: application/json' \  
  -d '{"model": "gemma3:1b-it-qat", "prompt": "Hello, world!"}'
```

Key Features

- **Model management:** `list/pull` — Install and switch among Llama 3.3, DeepSeek-R1, Gemma 3, Mistral, Phi-4, and more.
- **Local inference:** `run` — Execute prompts entirely on-device for privacy and zero latency beyond hardware limits.
- **Persistent server:** `serve` — Expose a local REST API for multi-session chats and integration into scripts or apps.
- **Version pinning:** `pull model:tag` — Pin exact model versions for reproducible demos and experiments.
- **Resource control:** `--threads / --context` — Tune CPU/GPU usage and maximum context window for performance and memory management.

Real-World Use Cases

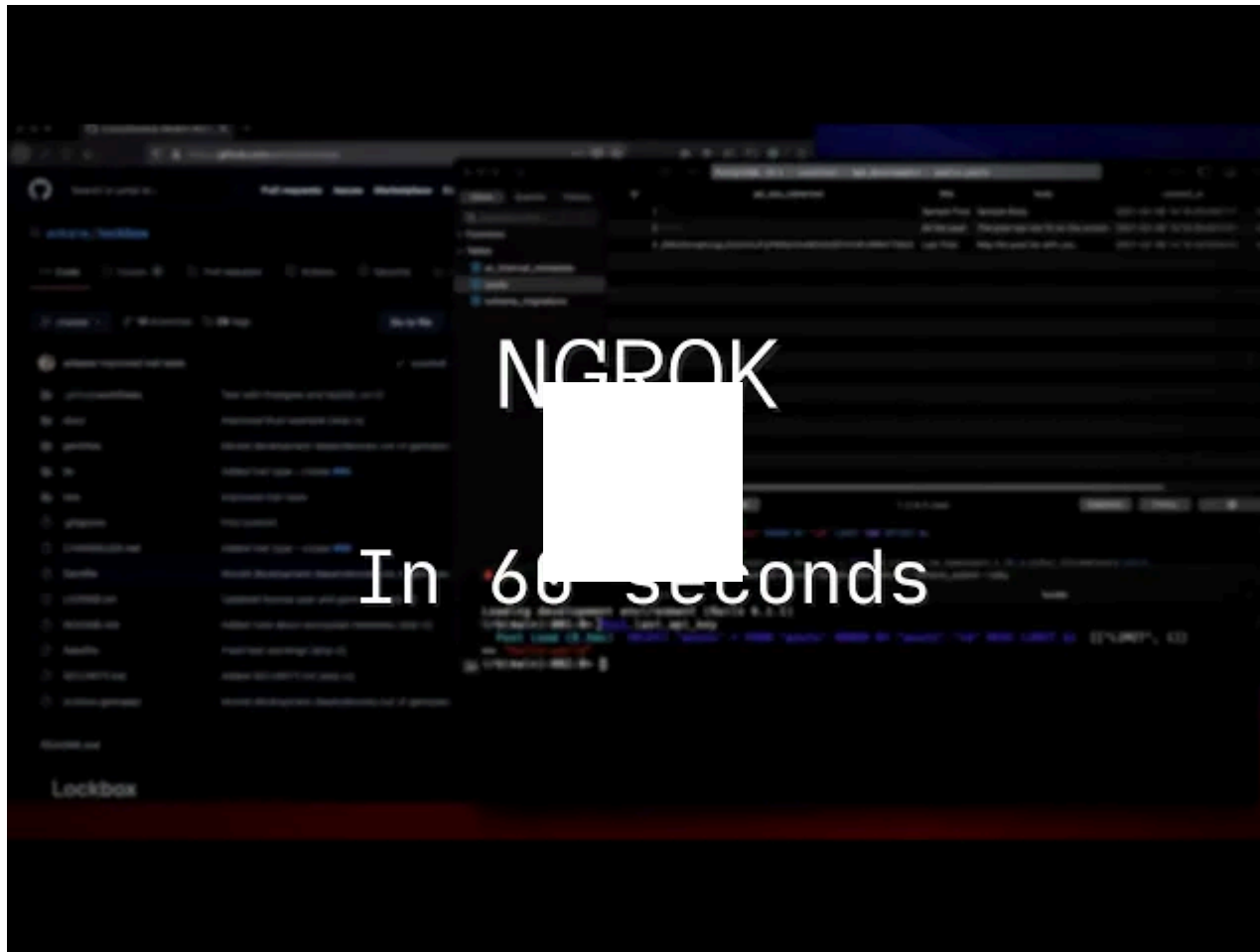
- **Quick prototyping.** Brainstorm slide decks or blog outlines offline, without worrying about API quotas: `ollama run gemma-3 'Outline a slide deck on Agile best practices'`
- **Data privacy.** Summarize sensitive documents on-device, retaining full control of your data: `cat financial_report.pdf | ollama run phi-4 'Summarize the key findings'`
- **CI/CD integration.** Validate PR descriptions or test YAML configurations in your pipeline without incurring API costs: `git diff origin/main | ollama run llama2 'Check for style and clarity issues'`
- **Local app embedding.** Power a desktop or web app via the local REST API for instant LLM features: `curl -X POST http://localhost:11434/api/chat -d '{"model": "mistral", "prompt": "Translate to German"}'`

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Tunneling: ngrok

[Ngrok](#) is a tool that creates secure tunnels to your localhost, making your local development server accessible to the internet. It's essential for testing webhooks, sharing work in progress, or debugging applications in production-like environments.



Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

To get started, log into ngrok.com and [get an authtoken from the dashboard](#). Copy it. Then run:

```
uvx ngrok config add-authtoken $YOUR_AUTHTOKEN
```

Now you can forward any local port to the internet. For example:

```
# Start a local server on port 8000
```

```
uv run -m http.server 8000
```

```
# Start HTTP tunnel
```

```
uvx ngrok http 8000
```

Here are useful things you can do with `ngrok http`:

- `ngrok http file://.` to serve local files
- `--response-header-add "Access-Control-Allow-Origin: *"` to enable CORS
- `--oauth google --oauth-client-id $CLIENT_ID --oauth-client-secret $SECRET --oauth-allow-domain example.com --oauth-allow-email user@example.org` to restrict users to `@example.com` and [user@example.org](#) using Google Auth
- `--ua-filter-deny ".*bot$"` to reject user agents ending with `bot`

Ask AI

- What is Ollama, how do I run a local model, and how do I call its API from another program?
- How do I choose a model size that fits my CPU, GPU, RAM, and latency constraints?
- How do `OLLAMA_HOST`, `OLLAMA_ORIGINS`, and tunnels like ngrok affect browser or remote access to Ollama?

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

esnopCo operates a local Ollama instance (`http://localhost:11434`) to power our internal AI chat assistant. To allow remote diagnostics and frontend integration, you need to:

1. **Enable CORS** for Ollama. Set the environment variable `OLLAMA_ORIGINS="*"`. Then run Ollama. For example:

```
export OLLAMA_ORIGINS="*"
ollama serve
```

2. **Expose** it via ngrok, injecting your email in the header:

```
ngrok http 11434 --response-header-add "X-Email: 23f3002123@ds.study.iitm.ac.in" --response-header-add 'Access-
```



3. Note the HTTPS forwarding URL that ngrok prints (e.g. `https://abcd1234.ngrok-free.app`).

4. **Verify** via a simple fetch (or curl) that:

- CORS header `Access-Control-Allow-Origin: *` is present
- Your `X-User-Email` header is echoed
- The JSON body looks like a valid Ollama response

Paste your ngrok forwarding URL here:

https://abcd1234.ngrok-free.app

We'll automatically fetch and verify the response headers and body.

Check

Discuss

 Ask AI

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Terminal: Bash

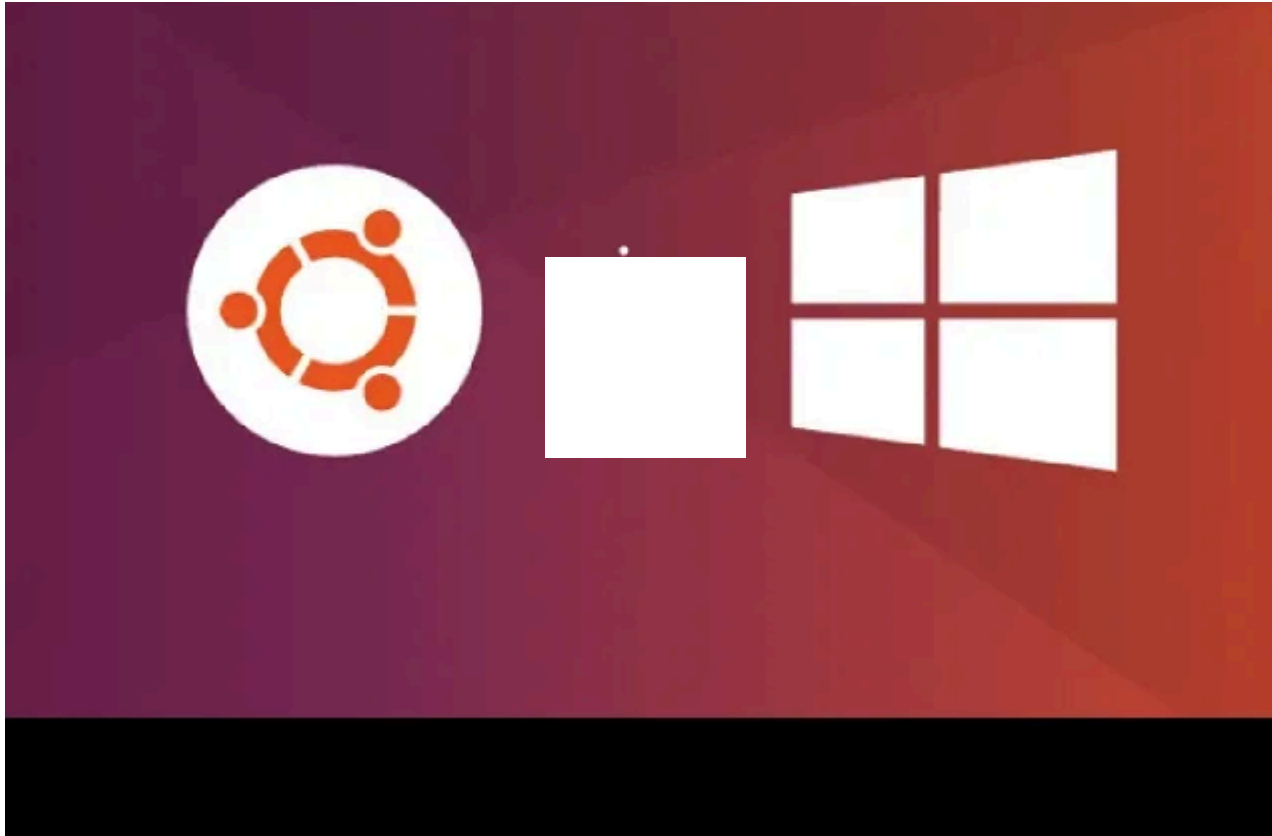
UNIX shells are the de facto standard in the data science world and [Bash](#) is the most popular. This is available by default on Mac and Linux.

On Windows, install [Git Bash](#) or [WSL](#) to get a UNIX shell.

Watch this video to install WSL (12 min).

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



Watch this video to understand the basics of Bash and UNIX shell commands (75 min).

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Essential Commands:

```
# File Operations
ls -la           # List all files with details
cd path/to/dir   # Change directory
pwd              # Print working directory
cp source dest   # Copy files
mv source dest   # Move/rename files
rm -rf dir        # Remove directory recursively
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
awk '{print $1}' f    # Process text by columns
cat file | wc -l     # Count lines
```

Process Management

```
ps aux              # List processes
kill -9 PID         # Force kill process
top                 # Monitor processes
htop                # Interactive process viewer
```

Network

```
curl url            # HTTP requests
wget url            # Download files
nc -zv host port    # Test connectivity
ssh user@host       # Remote login
```

Count unique values in CSV column

```
cut -d',' -f1 data.csv | sort | uniq -c
```

Quick data analysis

```
awk -F',' '{sum+=$2} END {print sum/NR}' data.csv # Average
sort -t',' -k2 -n data.csv | head               # Top 10
```

Monitor log in real-time

```
tail -f log.txt | grep --color 'ERROR'
```

Bash Scripting Essentials:

```
#!/bin/bash
```

Variables

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
# Loops
for i in {1..5}; do
    echo $i
done

# Conditionals
if [ -f "file.txt" ]; then
    echo "File exists"
fi

# Functions
process_data() {
    local input=$1
    echo "Processing $input"
}
```

Productivity Tips:

1. Command History

| | |
|----------------------|------------------------|
| <code>history</code> | # Show command history |
| <code>Ctrl+R</code> | # Search history |
| <code>!!</code> | # Repeat last command |
| <code>!\$</code> | # Last argument |

2. Directory Navigation

| | |
|------------------------|----------------------------|
| <code>pushd dir</code> | # Push directory to stack |
| <code>popd</code> | # Pop directory from stack |

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

3. JOB CONTROL

```
command &      # Run in background
Ctrl+Z         # Suspend process
bg             # Resume in background
fg             # Resume in foreground
```

4. Useful Aliases - typically added to ~/.bashrc

```
alias ll='ls -la'
alias gs='git status'
alias jupyter='jupyter notebook'
alias activate='source venv/bin/activate'
```

Ask AI

- What command-line tools help with bulk text replacement across many files, and when should I use sed, perl, or ripgrep?
- How do I safely preview a replacement before modifying files in place?
- How do I handle regex metacharacters, path filters, and backups in a bulk replace task?

Download [q-replace-across-files.zip](#) and unzip it into a new folder, then replace all "IITM" (in upper, lower, or mixed case) with "IIT Madras" in **all files**. Leave everything as-is - don't change the [line endings](#).

What does running `cat * | sha256sum` in that folder show in bash?

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Check

Discuss

 Ask AI

20 Sort and Filter a JSON Product Catalog (0.5 marks)

JSON

JSON (JavaScript Object Notation) is the de facto standard format for data exchange on the web and APIs. Its human-readable format and widespread support make it essential for data scientists working with web services, APIs, and configuration files.

For data scientists, JSON is essential when:

- Working with REST APIs and web services
- Storing configuration files and metadata
- Parsing semi-structured data from databases like MongoDB
- Creating data visualization specifications (e.g., Vega-Lite)

Watch this comprehensive introduction to JSON (15 min):

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



Key concepts to understand in JSON:

- JSON only supports 6 data types: strings, numbers, booleans, null, arrays, and objects
- You can nest data. Arrays and objects can contain other data types, including other arrays and objects
- Always validate. Ensure JSON is well-formed. Common errors: trailing commas, missing quotes, and incorrect escaping

[JSON Lines](#) is a format that allows you to store multiple JSON objects in a single line. It's useful for logging and streaming data.

Tools you could use with JSON:

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- [JSON Schema](#): Define the structure of your JSON data
- [jq](#): Command-line JSON processor

Common Python operations with JSON:

```
import json

# Parse JSON string
json_str = '{"name": "Alice", "age": 30}'
data = json.loads(json_str)

# Convert to JSON string
json_str = json.dumps(data, indent=2)

# Read JSON from file
with open('data.json') as f:
    data = json.load(f)

# Write JSON to file
with open('output.json', 'w') as f:
    json.dump(data, f, indent=2)

# Read JSON data into a Pandas DataFrame. JSON data is typically stored as an array of objects.
import pandas as pd
df = pd.read_json('data.json')

# Read JSON lines from file into a DataFrame. JSON lines are typically one line per object.
df = pd.read_json('data.jsonl', lines=True)
```

Practice JSON skills with these resources:

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- [JSON Schema validator](#), validate JSON against schemas

Ask AI

- How do I filter a JSON array by multiple conditions and then sort the matching records?
- When should I use jq, JavaScript, or Python for JSON transformation tasks?
- How do I avoid mistakes with numeric vs string sorting and missing JSON fields?

You're auditing an e-commerce catalog. Below is a JSON array of **100** products. Each has a **category**, **price**, and **name**.

1. Filter out any product with **price** < **82.00**.
2. Sort the remaining items by:
 - **category** (A→Z)
 - then **price** (highest→lowest)
 - then **name** (A→Z)
3. Paste the final array as a single minified JSON string below.

```
[{"category":"Apparel","price":94.01,"name":"Mini Gadget"}, {"category":"Home","price":47.72,"name":"Super Tool"}, {"category":"Electronics","price":184.13,"name":"Ultra Kit"}, {"category":"Apparel","price":134.2,"name":"Ultra Kit"}, {"category":"Apparel","price":135.42,"name":"Ultra Tool"}, {"category":"Apparel","price":177.45,"name":"Ultra Set"}, {"category":"Books","price":162.42,"name":"Ultra Set"}, {"category":"Home","price":51.92,"name":"Smart Widget"}, {"category":"Electronics","price":67.49,"name":"Ultra Device"}, {"category":"Home","price":158.7,"name":"Smart Kit"}, {"category":"Books","price":191.85,"name":"Mini Item"}, {"category":"Apparel","price":112.73,"name":"Pro Device"}, {"category":"Electronics","price":59.94,"name":"Super Gadget"}, {"category":"Home","price":71.87,"name":"Eco Gadget"}, {"category":"Toys","price":170.22,"name":"Pro Widget"}, {"category":"Home","price":85.46,"name":"Ultra
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
Widget"}, {"category": "Books", "price": 198.94, "name": "Pro Tool"},
{"category": "Electronics", "price": 170.2, "name": "Super Kit"}, {"category": "Apparel", "price": 29.78, "name": "Mini
Widget"}, {"category": "Books", "price": 172.15, "name": "Pro Set"},
{"category": "Electronics", "price": 95.36, "name": "Smart Kit"}, {"category": "Electronics", "price": 162.89, "name": "Eco
Gadget"}, {"category": "Apparel", "price": 37.57, "name": "Pro Widget"},
{"category": "Electronics", "price": 50.48, "name": "Smart Kit"}, {"category": "Toys", "price": 59.98, "name": "Pro Kit"},
{"category": "Toys", "price": 54.61, "name": "Smart Set"}, {"category": "Apparel", "price": 27.96, "name": "Smart Device"},
{"category": "Home", "price": 50.44, "name": "Super Gadget"}, {"category": "Apparel", "price": 21.42, "name": "Mini Tool"},
{"category": "Home", "price": 132.55, "name": "Smart Tool"}, {"category": "Home", "price": 83.01, "name": "Mini Gadget"},
{"category": "Apparel", "price": 132.11, "name": "Pro Gadget"}, {"category": "Apparel", "price": 144.09, "name": "Deluxe
Widget"}, {"category": "Home", "price": 135.95, "name": "Ultra Widget"}, {"category": "Books", "price": 117.48, "name": "Mini
Tool"}, {"category": "Books", "price": 27.97, "name": "Smart Widget"},
{"category": "Apparel", "price": 112.58, "name": "Ultra Widget"}, {"category": "Home", "price": 170.94, "name": "Pro Item"},
{"category": "Toys", "price": 186.98, "name": "Eco Widget"}, {"category": "Books", "price": 121.91, "name": "Deluxe Item"},
{"category": "Home", "price": 68.4, "name": "Smart Item"}, {"category": "Electronics", "price": 45.87, "name": "Eco
Gadget"}, {"category": "Electronics", "price": 41.59, "name": "Eco Item"},
{"category": "Electronics", "price": 49.7, "name": "Eco Gadget"}, {"category": "Books", "price": 99.86, "name": "Ultra
Widget"}, {"category": "Apparel", "price": 45.73, "name": "Eco Kit"}, {"category": "Toys", "price": 112.08, "name": "Deluxe
Widget"}, {"category": "Apparel", "price": 35.91, "name": "Pro Device"},
{"category": "Electronics", "price": 179.94, "name": "Smart Gadget"}, {"category": "Home", "price": 103.46, "name": "Ultra
Set"}, {"category": "Toys", "price": 32.13, "name": "Smart Widget"}, {"category": "Toys", "price": 187.03, "name": "Mini
Item"}, {"category": "Apparel", "price": 158.9, "name": "Smart Tool"}, {"category": "Books", "price": 193.56, "name": "Mini
Set"}, {"category": "Books", "price": 181.42, "name": "Eco Widget"}, {"category": "Toys", "price": 116.65, "name": "Deluxe
Kit"}, {"category": "Books", "price": 171.91, "name": "Ultra Gadget"}, {"category": "Toys", "price": 78.47, "name": "Deluxe
Device"}, {"category": "Toys", "price": 59.66, "name": "Eco Device"},
{"category": "Electronics", "price": 112.42, "name": "Super Set"}, {"category": "Apparel", "price": 169.15, "name": "Mini
Tool"}, {"category": "Home", "price": 160.42, "name": "Super Kit"}, {"category": "Apparel", "price": 51.17, "name": "Deluxe
Device"}, {"category": "Books", "price": 124.11, "name": "Super Item"}, {"category": "Home", "price": 89.93, "name": "Eco
Device"}, {"category": "Books", "price": 133.08, "name": "Pro Device"}, {"category": "Books", "price": 20.85, "name": "Smart
Widget"}, {"category": "Electronics", "price": 164.54, "name": "Smart Set"},
{"category": "Toys", "price": 75.29, "name": "Mini Item"}, {"category": "Toys", "price": 60.18, "name": "Ultra Set"},
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
Item"}, {"category": "Toys", "price": 41.47, "name": "Deluxe Device"}, {"category": "Home", "price": 102.96, "name": "Pro Set"}, {"category": "Electronics", "price": 172.63, "name": "Pro Device"}, {"category": "Electronics", "price": 178.39, "name": "Deluxe Item"}, {"category": "Toys", "price": 169.65, "name": "Deluxe Tool"}, {"category": "Electronics", "price": 182.85, "name": "Super Widget"}, {"category": "Books", "price": 50.78, "name": "Ultra Widget"}, {"category": "Home", "price": 145.77, "name": "Pro Device"}, {"category": "Apparel", "price": 51.1, "name": "Eco Item"}, {"category": "Toys", "price": 51.54, "name": "Eco Kit"}, {"category": "Toys", "price": 156.21, "name": "Smart Device"}, {"category": "Home", "price": 120.12, "name": "Super Tool"}, {"category": "Books", "price": 133.99, "name": "Ultra Item"}, {"category": "Books", "price": 187.51, "name": "Pro Item"}, {"category": "Electronics", "price": 62.31, "name": "Pro Widget"}, {"category": "Apparel", "price": 93.63, "name": "Deluxe Set"}, {"category": "Electronics", "price": 140.47, "name": "Super Widget"}, {"category": "Home", "price": 142.25, "name": "Mini Set"}, {"category": "Toys", "price": 59.38, "name": "Super Tool"}, {"category": "Home", "price": 87.65, "name": "Pro Kit"}]
```

Sorted & filtered JSON:

Check

Discuss

 Ask AI

21 SQL: Average salary by department (0.5 marks)

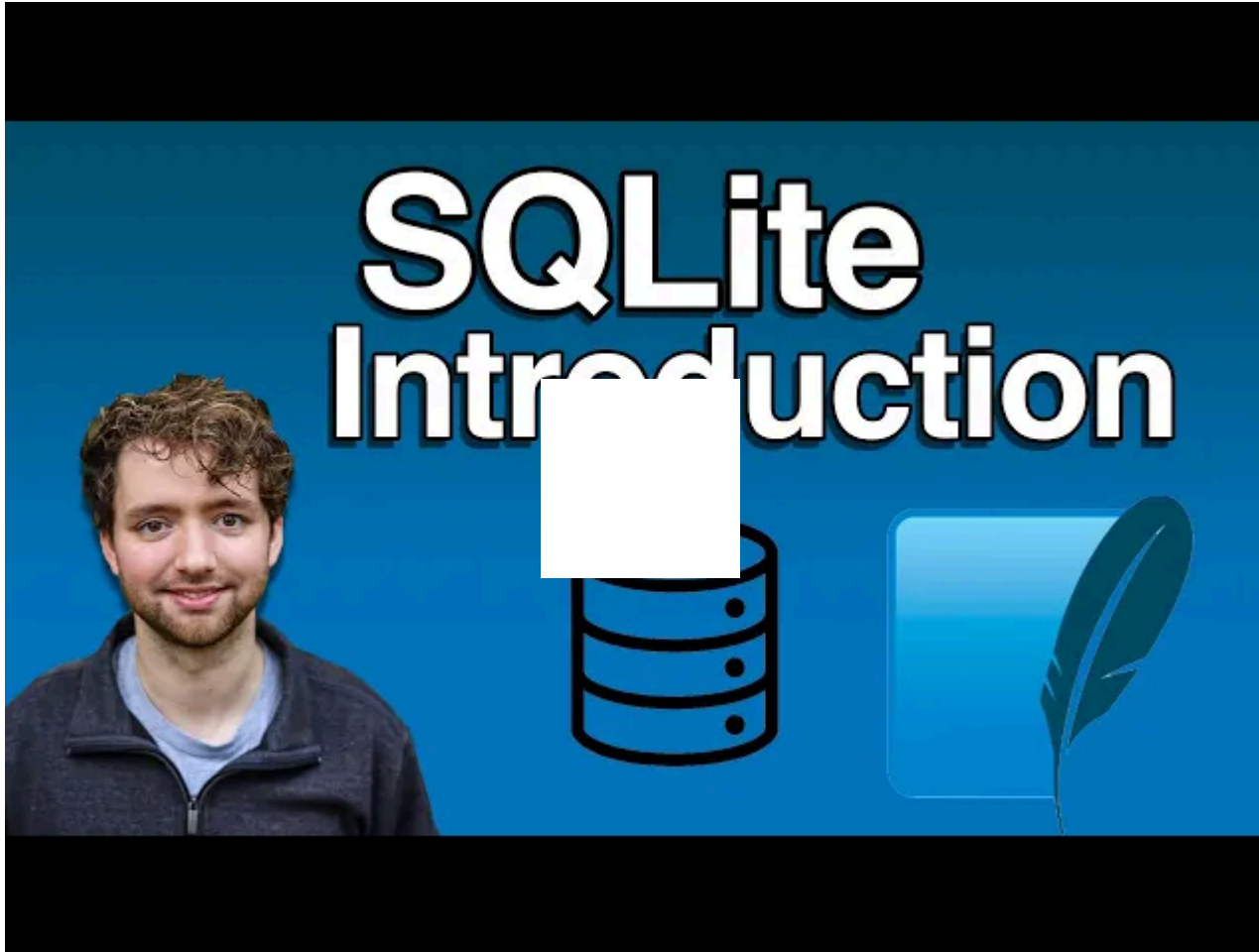
Database: SQLite

Relational databases are used to store data in a structured way. You'll often access databases created by others for analysis.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Watch these introductory videos to understand SQLite and how it's used in Python (34 min):



Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

There are many non-relational databases (NoSQL) like [ElasticSearch](#), [MongoDB](#), [Redis](#), etc. that you should know about and we may cover later.

Core Concepts:

```
-- Create a table
CREATE TABLE users (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT UNIQUE,
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

```
-- Insert data
INSERT INTO users (name, email) VALUES
    ('Alice', 'alice@example.com'),
    ('Bob', 'bob@example.com');

-- Query data
SELECT name, COUNT(*) as count
FROM users
GROUP BY name
HAVING count > 1;

-- Join tables
SELECT u.name, o.product
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE o.status = 'pending';
```

Python Integration:

```
import sqlite3
from pathlib import Path
import pandas as pd

def query_database(db_path: Path, query: str) -> pd.DataFrame:
    """Execute SQL query and return results as DataFrame."""
    with sqlite3.connect(db_path) as conn:
        return pd.read_sql_query(query, conn)

# Example usage
db = Path('data.db')
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

```
GROUP BY date  
'')
```

Common Operations:

1. Database Management

```
-- Backup database  
.backup 'backup.db'  
  
-- Import CSV  
.mode csv  
.import data.csv table_name  
  
-- Export results  
.headers on  
.mode csv  
.output results.csv  
SELECT * FROM table;
```

2. Performance Optimization

```
-- Create index  
CREATE INDEX idx_user_email ON users(email);  
  
-- Analyze query  
EXPLAIN QUERY PLAN  
SELECT * FROM users WHERE email LIKE '%@example.com';
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

3. Data Analysis

```
-- Time series aggregation
SELECT
    date(timestamp),
    COUNT(*) as events,
    AVG(duration) as avg_duration
FROM events
GROUP BY date(timestamp);

-- Window functions
SELECT *,
    AVG(amount) OVER (
        PARTITION BY user_id
        ORDER BY date
        ROWS BETWEEN 3 PRECEDING AND CURRENT ROW
    ) as moving_avg
FROM transactions;
```

Tools to work with SQLite:

- [SQLiteStudio](#): Lightweight GUI
- [DBeaver](#): Full-featured GUI
- [sqlite-utils](#): CLI tool
- [Datasette](#): Web interface

Ask AI

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

How do I check whether my SQL aggregation result is correct using a small hand-calculated example?

HR Analytics for GlobalTech Corporation

GlobalTech Corporation is a multinational technology company with over 10,000 employees across various departments. The Human Resources department is conducting a comprehensive compensation analysis to ensure pay equity, maintain competitive salary structures, and support budget planning for the upcoming fiscal year.

The HR team needs to analyze salary distributions across different departments to:

- **Ensure fair compensation:** Identify potential pay disparities between departments
- **Budget planning:** Understand departmental payroll costs for resource allocation
- **Competitive benchmarking:** Compare internal averages with industry standards
- **Strategic hiring:** Set appropriate salary ranges for new positions
- **Performance reviews:** Establish context for merit-based salary adjustments

This analysis will help the executive team make informed decisions about compensation strategies, departmental budgets, and organizational restructuring while maintaining employee satisfaction and market competitiveness.

Your Task

There is an `employees` table in a SQLite database with columns `employee_id`, `name`, `department`, and `salary`. Each row represents an employee in the company.

| employee_id | name | department | salary |
|-------------|--------------|-------------|--------|
| 2564 | Employee_895 | Finance | 54842 |
| 6450 | Employee_912 | Engineering | 46291 |

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

| | | | |
|------|--------------|-------------|--------|
| 5364 | Employee_882 | HR | 109704 |
| 5534 | Employee_804 | Engineering | 91210 |
| ... | | | |

What is the average salary for each department? Write SQL to calculate it.

Calculate the average salary for each department. Round the results to the nearest whole number. Order the results by department name alphabetically.

Check

Discuss

 Ask AI

22 Process files with different encodings (2 marks)

Unicode

Ever noticed when you copy-paste some text and get garbage symbols? Or see garbage when you load a CSV file? This video explains why. It covers how computers store text (called character encoding) and why it sometimes goes wonky.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Some programs try to guess encodings (sometimes badly!). A signature called BOM (Byte Order Mark) helps computers know exactly how to read text files correctly.

Learn how Unicode, UTF-8 and character encoding works. This is a common gotcha when building apps that handle international text - something bootcamps often skip but developers and data scientists regularly face in the real world.

Unicode is fundamental for data scientists working with international data. Here are key concepts you need to understand:

- **Character Encodings:** Different ways to represent text in computers
 - ASCII (7-bit): Limited to 128 characters, English-only
 - UTF-8: Variable-width encoding, backwards compatible with ASCII
 - UTF-16: Variable-width (2 or 4 bytes with surrogate pairs), used in Windows and Java
 - UTF-32: Fixed-width encoding, memory inefficient but simple

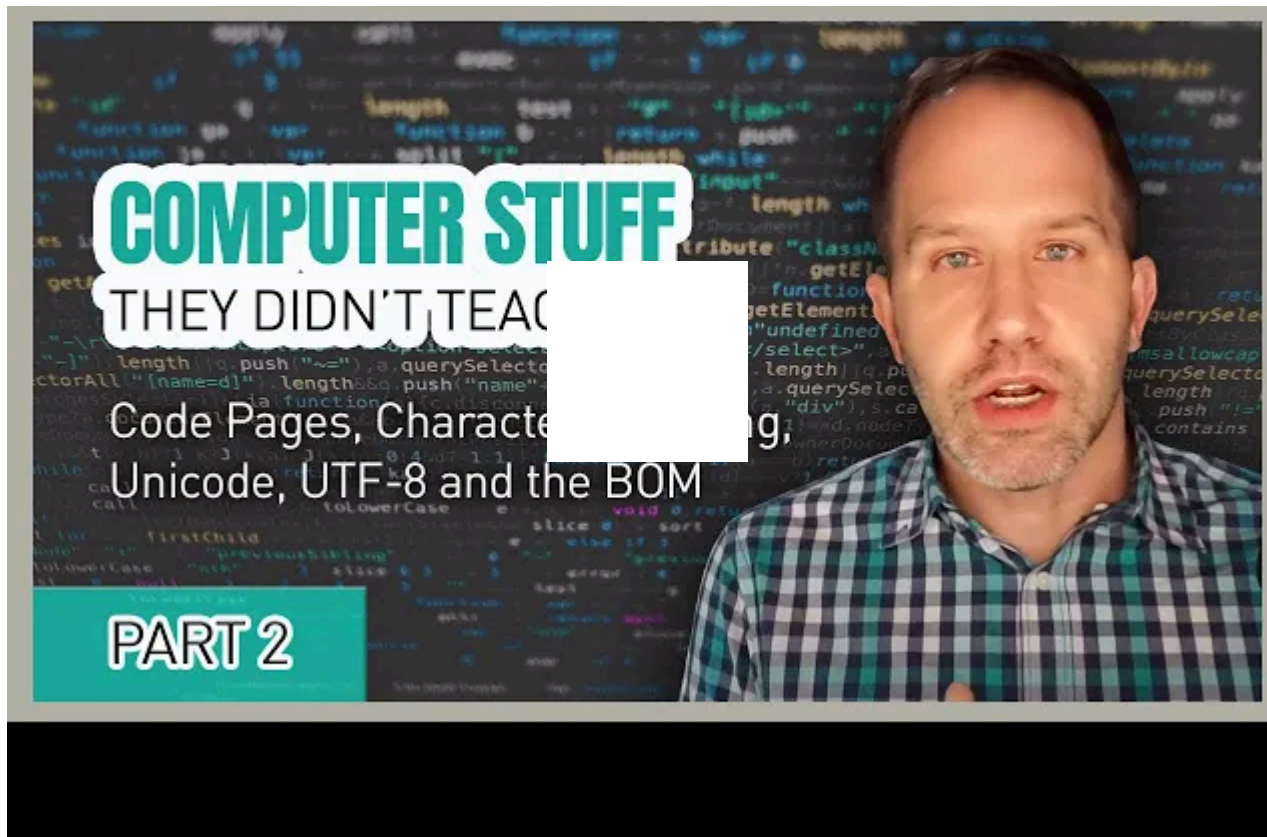
Common encoding issues you'll encounter:

```
# Reading files with explicit encoding
with open('file.txt', encoding='utf-8') as f:
    text = f.read()

# Handling encoding errors
import pandas as pd
df = pd.read_csv('data.csv', encoding='utf-8', errors='replace')

# Detecting file encoding
import chardet
with open('unknown.txt', 'rb') as f:
    result = chardet.detect(f.read())
print(result['encoding'])
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Ask AI

- What is Unicode normalization and why can identical-looking text compare as different strings?
 - How do I count Unicode code points, bytes, and characters without mixing them up?
 - How do I safely process international text in JavaScript, Python, spreadsheets, and command-line tools?
-

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- **data1.csv**: CSV file encoded in CP-1252
- **data2.csv**: CSV file encoded in UTF-8
- **data3.txt**: Tab-separated file encoded in UTF-16

Each file has 2 columns: symbol and value. Sum up all the values where the symbol matches ~ OR “ OR ‘ across all three files.

What is the sum of all values associated with these symbols?

Check

Discuss

 Ask AI

23 Use DevTools (1 mark)

Browser: DevTools

[Chrome DevTools](#) is the de facto standard for web development and data analysis in the browser. You'll use this a lot when debugging and inspecting web pages.

Here are the key features you'll use most:

1. Elements Panel

- Inspect and modify HTML/CSS in real-time
- Copy CSS selectors for web scraping
- Debug layout issues with the Box Model

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

2. Console Panel

- JavaScript REPL environment
- Log and debug data
- Common console methods:

```
console.table(data); // Display data in table format
console.group("Name"); // Group related logs
console.time("Label"); // Measure execution time
```

3. Network Panel

- Monitor API requests and responses
- Simulate slow connections
- Right-click on a request and select "Copy as fetch" to get the request.

4. Essential Keyboard Shortcuts

- **Ctrl+Shift+I** (Windows) / **Cmd+Opt+I** (Mac): Open DevTools
- **Ctrl+Shift+C**: Select element to inspect
- **Ctrl+L**: Clear console
- **\$0**: Reference currently selected element
- **\$\$('selector')**: Query selector all (returns array)

Videos from Chrome Developers (37 min total):

- [Fun & powerful: Intro to Chrome DevTools](#) (5 min)
- [Different ways to open Chrome DevTools](#) (5 min)
- [Faster DevTools navigation with shortcuts and settings](#) (3 min)

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- [TIME VS DOM: LET'S debug them](#) (5 min)
- [Caching demystified: Inspect, clear, and disable caches](#) (7 min)

Ask AI

- How do I inspect a web page with browser DevTools to find hidden inputs and element attributes?
- What are the Elements, Console, and Network tabs for, and which one should I check first for this task?
- How can I use the browser console to query the DOM and read an element value directly?

Just above this paragraph, there's a hidden input with a secret value.

What is the value in the hidden input?

Check

Discuss

 Ask AI

24 Use GitHub (1 mark)

Version Control: Git, GitHub

[Git](#) is the de facto standard for version control of software (and sometimes, data as well). It's a system that keeps track of changes you make to files and folders. It allows you to revert to a previous state, compare changes, etc. It's a central tool in any

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

with others, and provides many useful tools for developers.

Watch these introductory videos to learn the basics of Git and GitHub (98 min):



Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Essential Git Commands:

Repository Setup

```
git init                # Create new repo
git clone url           # Clone existing repo
git remote add origin url # Connect to remote
```

Basic Workflow

```
git status              # Check status
git add .               # Stage all changes
```

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Branching

| | |
|-------------------------|------------------------|
| git branch | # List branches |
| git checkout -b feature | # Create/switch branch |
| git merge feature | # Merge branch |
| git rebase main | # Rebase on main |

History

| | |
|--------------------------|-------------------------|
| git log --oneline | # View history |
| git diff commit1 commit2 | # Compare commits |
| git blame file | # Show who changed what |

Best Practices:

1. Commit Messages

Good commit message format

type(scope): summary

Detailed description of changes.

Examples

feat(api): add user authentication

fix(db): handle null values in query

2. Branching Strategy

- o main: Production code
- o develop: Integration branch
- o feature/*: New features

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

- Keep PRs small (<400 lines)
- Use draft PRs for WIP
- Review your own code first
- Respond to all comments

Essential Tools

- [GitHub Desktop](#): GUI client
- [GitLens](#): VS Code extension
- [gh](#): GitHub CLI
- [pre-commit](#): Git hooks

Ask AI

- How do I create a GitHub repository, commit a JSON file, and push it to the default branch?
- What is the difference between a GitHub page URL, a repository file URL, and a raw.githubusercontent.com URL?
- How do I verify that a raw GitHub URL returns valid JSON with the expected email field?

Let's make sure you know how to use GitHub. [Create a GitHub account](#) if you don't have one. Create a new public repository. Commit a single JSON file called `email.json` with the value `{"email": "23f3002123@ds.study.iitm.ac.in"}` and push it.

Enter the raw Github URL of `email.json` so we can verify it. (It will begin with `https://raw.githubusercontent.com/[GITHUB ID]/[REPO NAME]/...`)

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

25 Deploy a POST analytics endpoint to Vercel (3 marks)

Serverless hosting: Vercel

Serverless platforms let you rent a single function instead of an entire machine. They're perfect for small web tools that *don't need to run all the time*. Here are some common real-life uses:

- A contact form that emails you when someone wants to hire you (runs for 2-3 seconds, a few times per day)
- A tool that converts uploaded photos to black and white (runs for 5-10 seconds when someone uploads a photo)
- A chatbot that answers basic questions about your business hours (runs for 1-2 seconds per question)
- A newsletter sign-up that adds emails to your mailing list (runs for 1 second per sign-up)
- A webhook that posts your Etsy sales to Discord (runs for 1 second whenever you make a sale)

You only pay when someone uses your tool, and the platform automatically handles busy periods. For example, if 100 people fill out your contact form at once, the platform creates 100 temporary copies of your code to handle them all. When they're done, these copies disappear. It's cheaper than running a full-time server because you're not paying for the time when no one is using your tool - most tools are idle 95% of the time!

Rather than writing a full program, serverless platforms let you write functions. These functions are called via HTTP requests. They run in a cloud environment and are scaled up and down automatically. But this means you write programs in a different style. For example:

- You can't `pip install` packages - you have to use `requirements.txt`
- You can't read or write files from the file system - you can only use APIs.
- You can't run commands (e.g. `subprocess.run()`)

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Here's a [quickstart](#). [Sign up for Vercel](#). Create an empty `git` repo with this `api/index.py` file.

To deploy a FastAPI app, add a `requirements.txt` file with `fastapi` as a dependency.

```
fastapi
```

Add your FastAPI code to `api/index.py`.

```
# api/index.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}
```

Add a `vercel.json` file to the root of your repository.

```
{
  "builds": [{ "src": "api/index.py", "use": "@vercel/python" }],
  "routes": [{ "src": "/(.*)", "dest": "api/index.py" }]
}
```

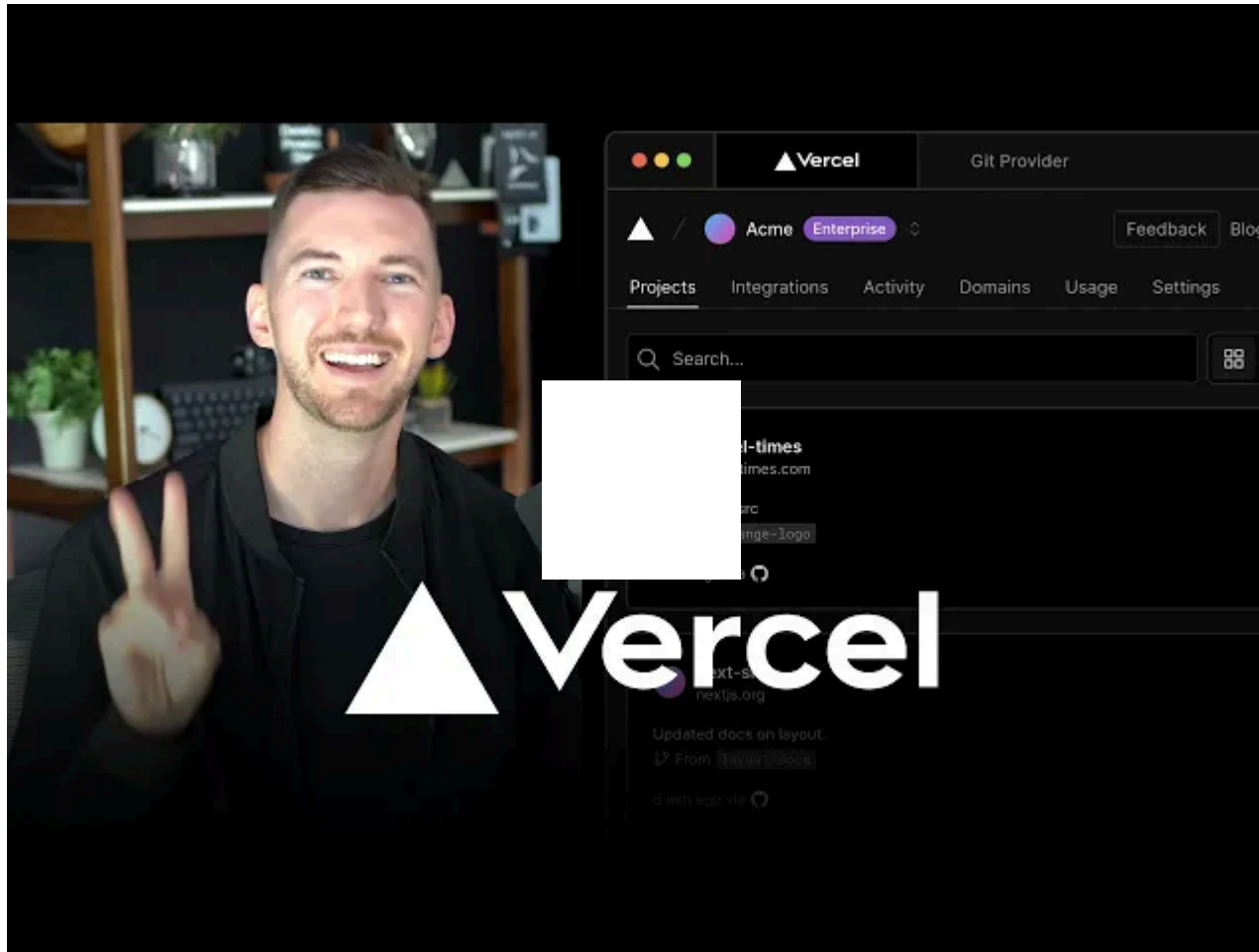
On the command line, run:

- `npx vercel` to deploy a test version
- `npx vercel --prod` to deploy to production

Due Sun, 11 Oct, 2026, 11:59 pm IST

[All scores](#)

Videos



Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores



Ask AI

- How do I deploy a Python API endpoint to Vercel and make sure the public URL accepts POST requests?
 - How do I compute average latency, p95 latency, uptime, and threshold breach counts from JSON telemetry?
 - What are the common Vercel gotchas for Python serverless functions: routing, cold starts, CORS, and request bodies?
-

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Download the sample telemetry bundle `q-vercel-latency.json` and deploy a Python endpoint on **Vercel**.

Your endpoint must:

- Accept a **POST** request with JSON body `{"regions": [...], "threshold_ms": 180}`
- Return per-region metrics: `avg_latency` (mean), `p95_latency` (95th percentile), `avg_uptime` (mean), and `breaches` (count of records above threshold)
- Enable CORS for **POST** requests from any origin

We'll send `{"regions":["apac","amer"],"threshold_ms":167}` to your endpoint and verify the response (order doesn't matter).

What is the POST endpoint URL?

`https://your-app.vercel.app/api/latency`

Check

Discuss

 Ask AI

Check all

Save

Save regularly. Your *last saved* submission will be evaluated.

Due Sun, 11 Oct, 2026, 11:59 pm IST

All scores

Best of luck!